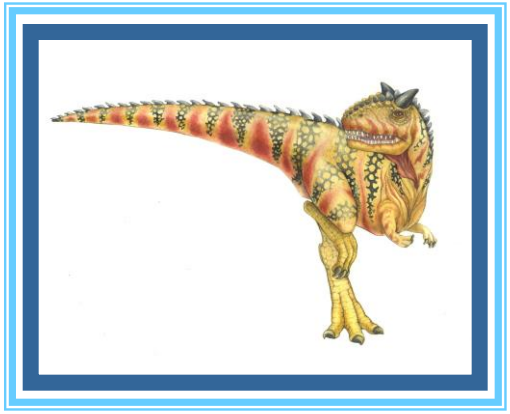
Chapter 3: Processes

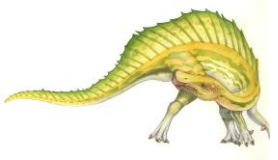


Silberschatz, Galvin and Gagne ©2018



Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Examples of IPC Systems
- Communication in Client-Server Systems



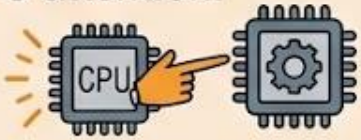
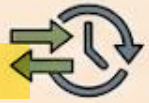
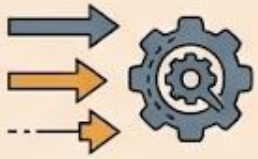


Objectives

- Identify the separate components of a process and illustrate how they are represented and scheduled in an operating system.
- Describe how processes are created and terminated in an operating system, including developing programs using the appropriate system calls that perform these operations.
- Describe and contrast interprocess communication using shared memory and message passing.
- Design programs that uses pipes and POSIX shared memory to perform interprocess communication.
- Describe client-server communication using sockets and remote procedure calls.
- Design kernel modules that interact with the Linux operating system.



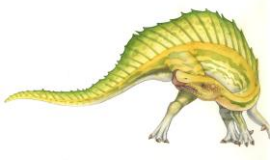
DIFFERENCE BETWEEN INTERRUPT, SYSTEM CALL, AND TRAP

Feature	INTERRUPT	SYSTEM CALL	TRAP
Meaning	An event that requests CPU attention. 	A request by a program for an OS service. 	A synchronous event that transfers control to a handler. 
Cause	Usually an external hardware event. 	An explicit request from from a program.  `read()` or `write()`	An instruction or an exceptional condition. `<1 breakpoint, <code __>` , or 
Nature	Usually asynchronous. 	Synchronous. 	Synchronous. 
Example	 Keyboard input, timer, network card.	`read()`, `write()`, `fork()`.	 Breakpoint, `<code`  exception.
Purpose	Handle external events. 	Access privileged OS services. 	Handle exceptions or deliberate control transfers. 



Process Concept

- An operating system executes a variety of programs that run as a process.
- **Process** – a program in execution; process execution must progress in sequential fashion. No parallel execution of instructions of a single process
- Multiple parts
  - The program code, also called **text section**
  - Current activity including **program counter**, processor registers
  - **Stack** containing temporary data
    - Function parameters, return addresses, local variables
  - **Data section** containing global variables
  - **Heap** containing memory dynamically allocated during run time



Process Concept

Anatomy of an OS Process

- OS executes programs as a Process.

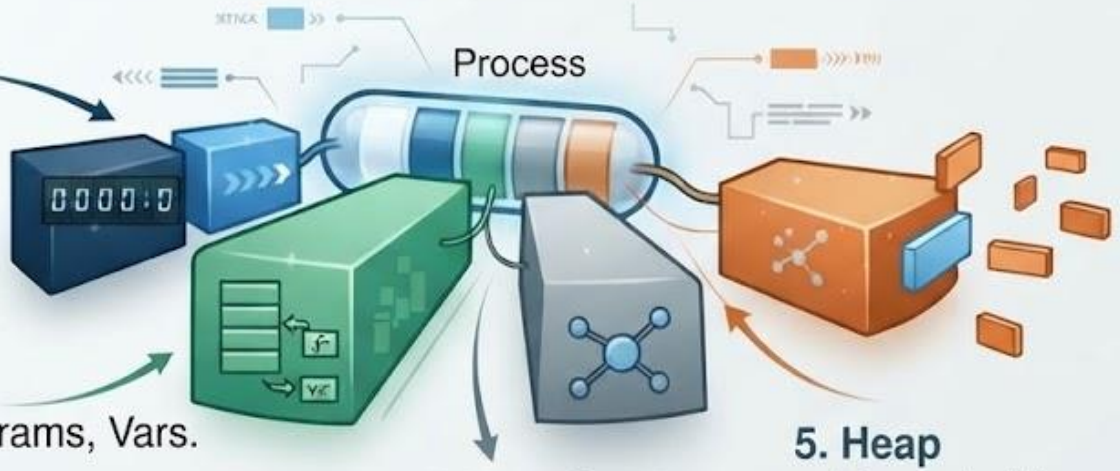


A program in execution: **Sequential execution.**

1. Text Section
Program Code.

2. Program Counter
Instruction Pointer.

3. Stack
Temp Data: Func Params, Vars.



4. Data Section
Global Variables.

5. Heap
Dynamic Memory.



Process Concept (Cont.)

Program vs. Process: Key Concepts



Program: PASSIVE Entity

Stored on disk (executable file).



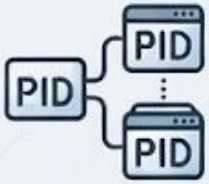
Process: ACTIVE Entity

Loaded into memory and executing.

- **Execution and Lifecycle**



Execution started via GUI clicks, command line entry, etc.



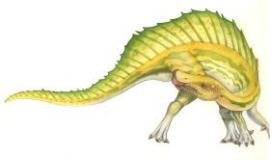
Multiple instances of the same program can run as separate processes.

Key Takeaway: Program is a blueprint; Process is the running execution of that blueprint.



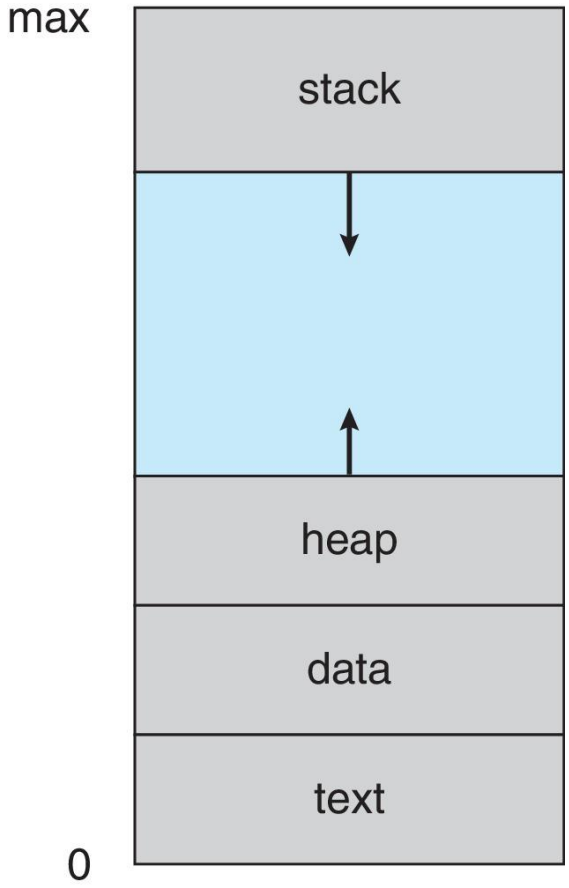
Process Concept (Cont.)

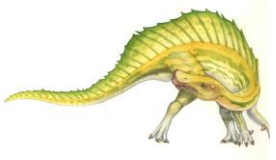
- Program is **passive** entity stored on disk (**executable file**); process is **active**
  - Program becomes process when an executable file is loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- One program can be several processes
  - Consider multiple users executing the same program





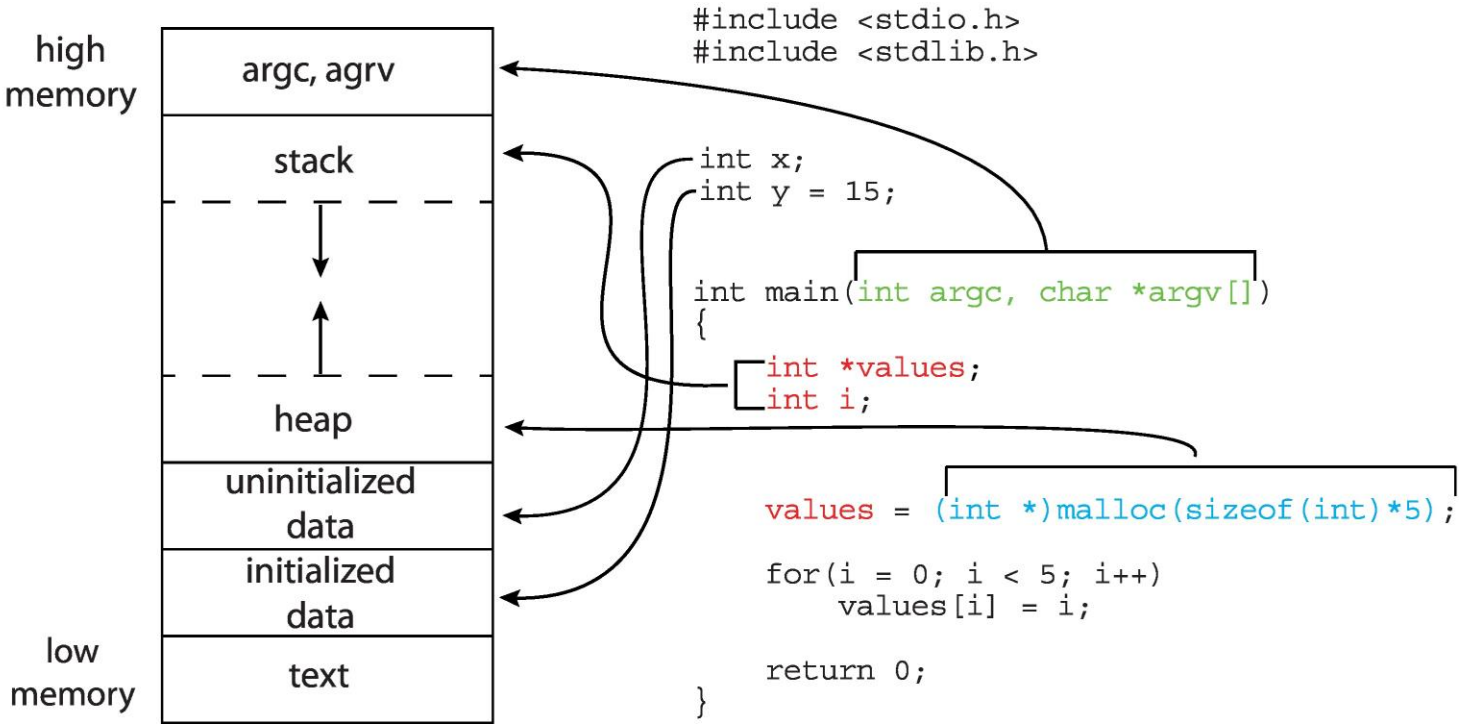
Process in Memory

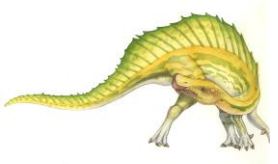






Memory Layout of a C Program







Process State

- As a process executes, it changes **state**

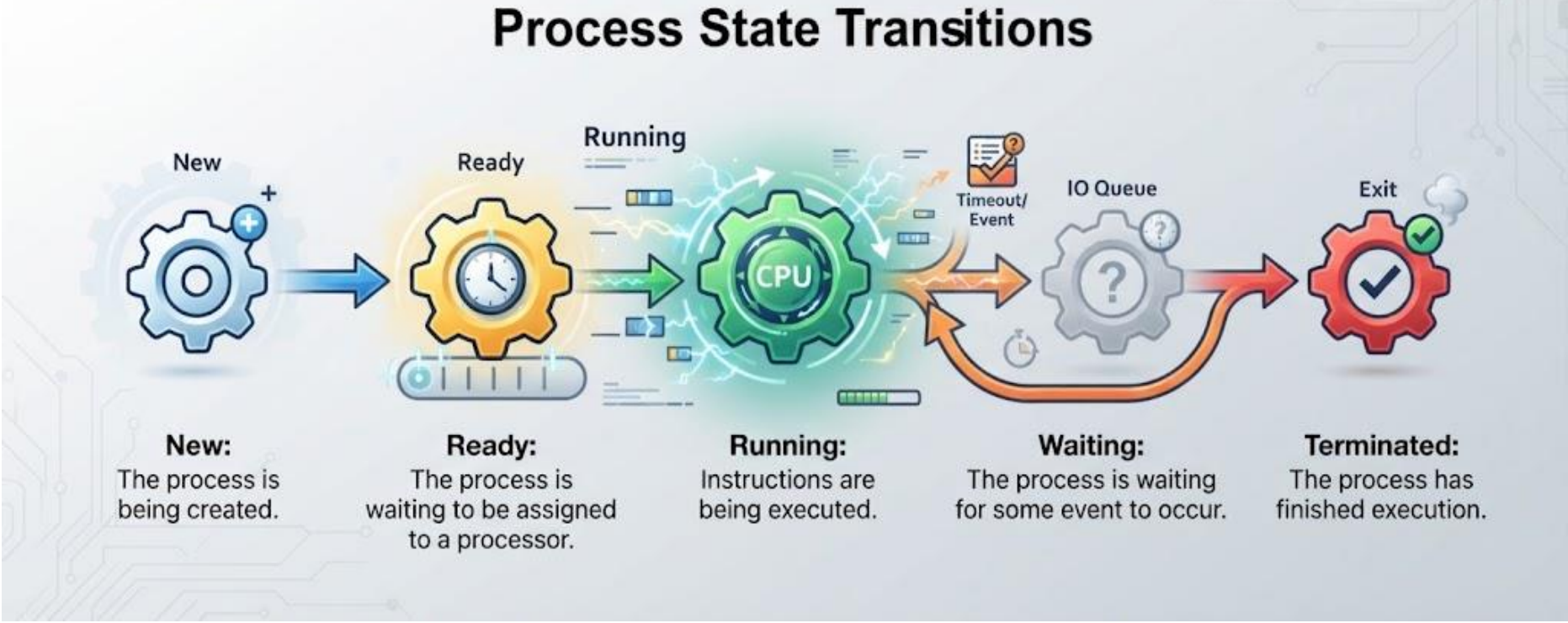
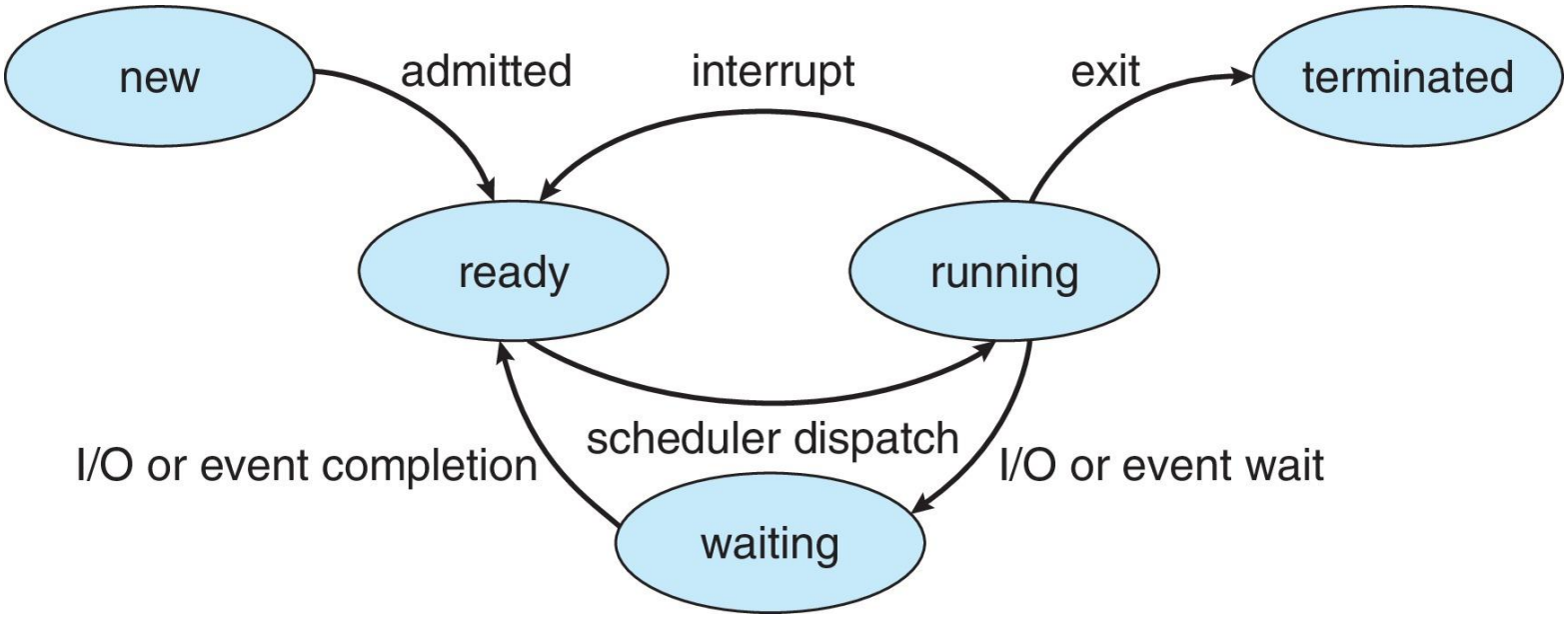
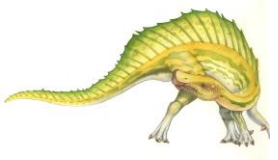






Diagram of Process State

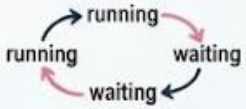






Process Control Block (PCB)

- **Process state** – running, waiting, etc.

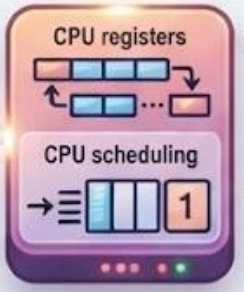


- **Program counter** – location of instruction to next execute.
- **Accounting start** – location of accounty elpsed start...
- **I/O status information** – I/O devices allocated to process, list of open files.









CPU registers & scheduling

- **CPU registers** – state information must be saved, ...
- **CPU scheduling information-** priorities... priorities...



- **Memory-management** information – memory alloc allocated to the process.



- **Accounting information** – CPU used, clock time elapsed since start...

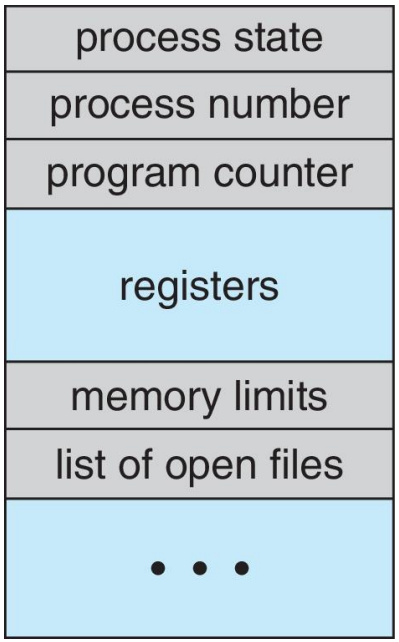
The PCB simply serves as the **repository** for all the data needed to start, or restart, a process, along with some accounting data.

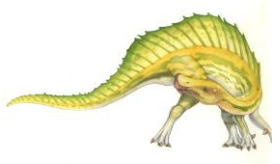


Process Control Block (PCB)

Information associated with each process(also called **task control block**)

- Process state – running, waiting, etc.
- Program counter – location of instruction to next execute
- CPU registers – state information must be saved to continue correctly after it is rescheduled.
  CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files
- The PCB simply serves as the repository for all the data needed to start, or restart, a process, along with some accounting data.



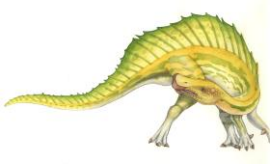




Process and Thread

Process:
A process can be thought of as a program in execution.

Thread:
A thread is the unit of execution within a process. A process can have anywhere from just one thread to many threads.





Process and Thread

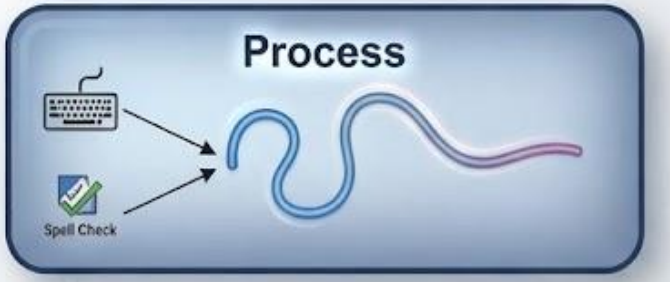
Process vs. Multithreaded Process: Evolution

Single-Threaded Process (Legacy)

Extends Concept

Multithreaded Process (Modern)

Single Thread of Execution



Performs one task at a time.
(e.g., typing OR spell checking).
Cannot run in parallel.

Multiple Threads of Execution



Performs more than one task at a time.
(e.g., typing AND spell checking).
Can run in parallel on multicore systems.



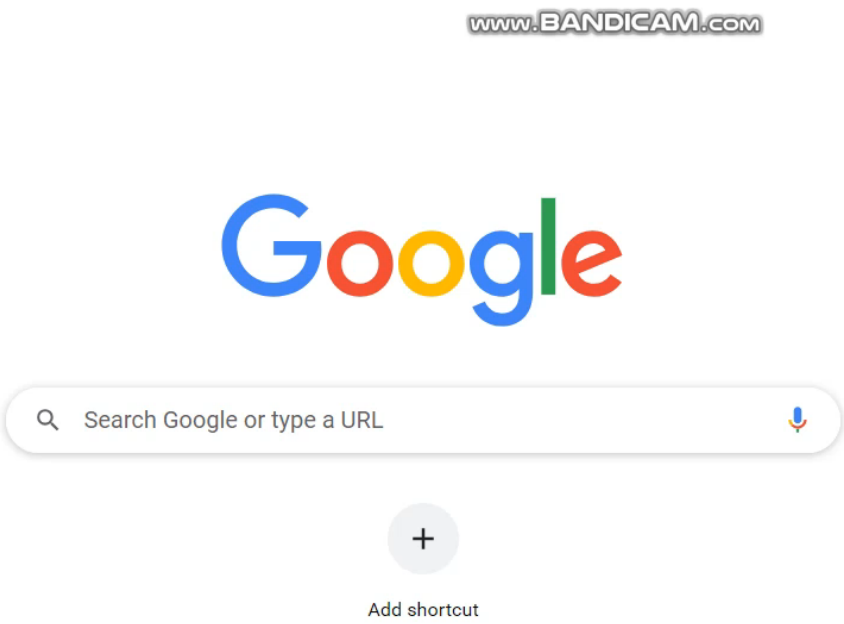
PCB Expanded:
Includes information for each thread.
Systems must support thread management.

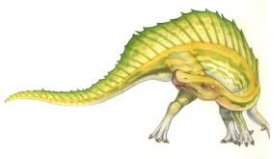
Operating System Concepts - Chapter 4 Introduction





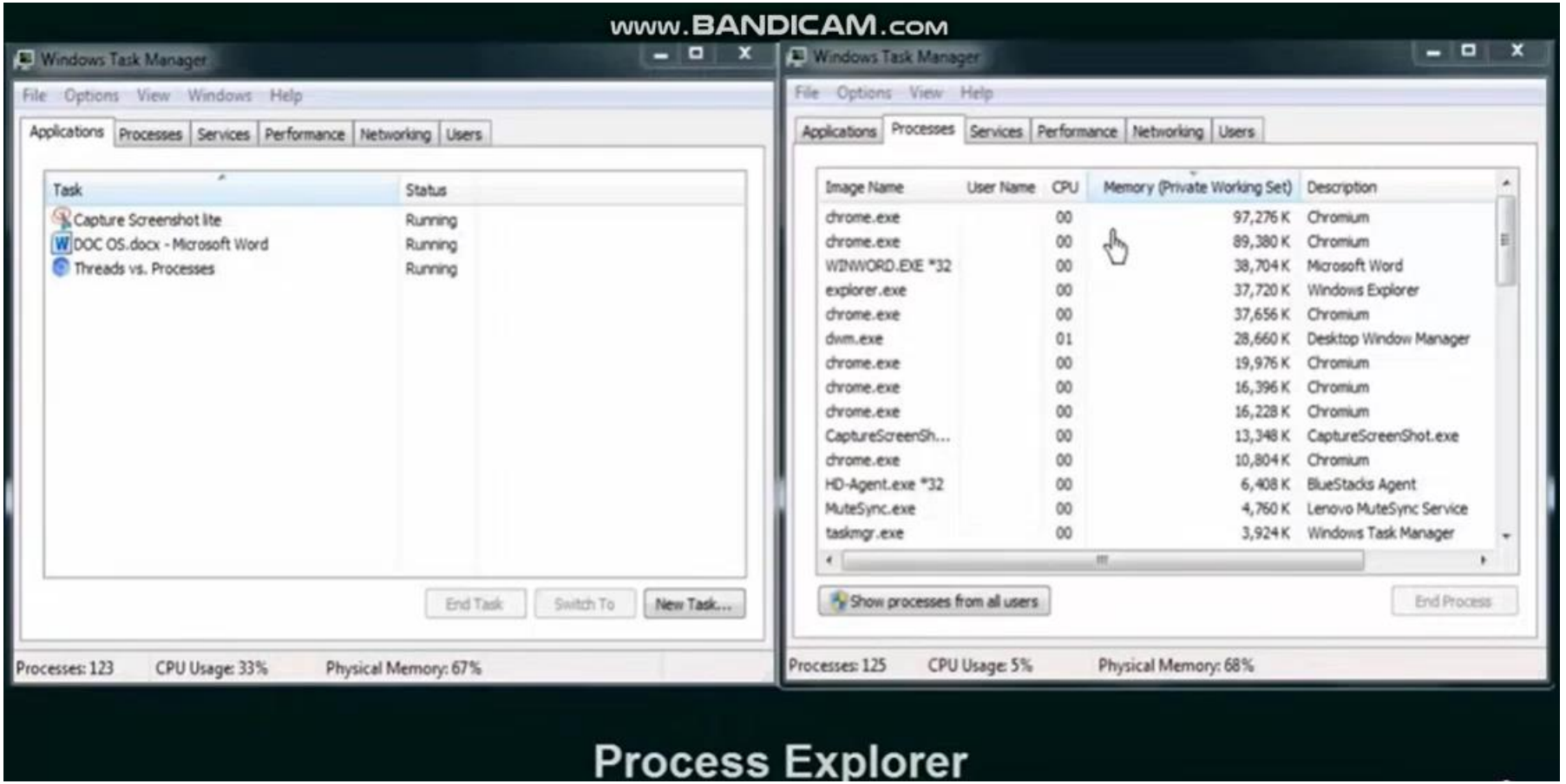
Thread example (video file)

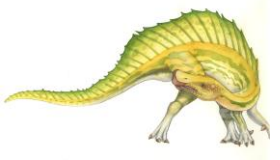






Threads example cont…

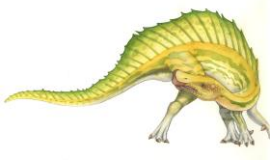






Threads

- So far, process has a single thread of execution
- Consider having multiple program counters per process
  - Multiple locations can execute at once
    - Multiple threads of control -> **threads**
- Must then have storage for thread details, multiple program counters in PCB
- Explore in detail in Chapter 4

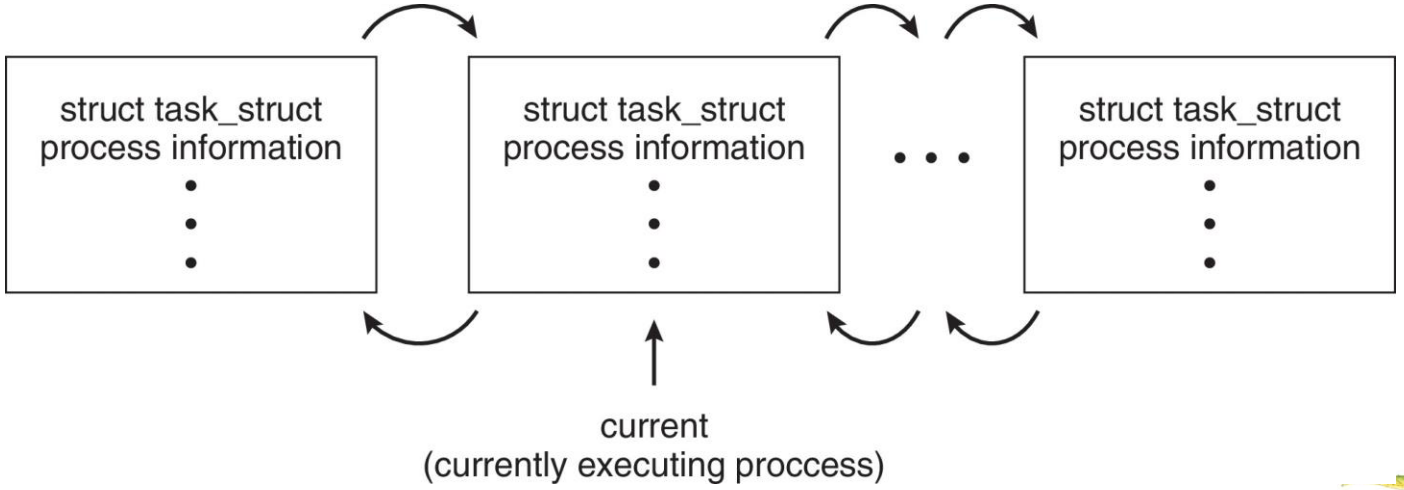


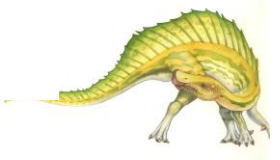


Process Representation in Linux

Represented by the C structure `task_struct`

```
pid t_pid;                    /* process identifier */
long state;                   /* state of the process */
unsigned int time_slice       /* scheduling information */
struct task_struct *parent;/* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files;/* list of open files */
struct mm_struct *mm;         /* address space of this
process */
```

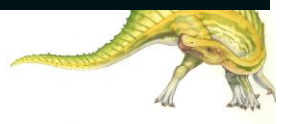






Process Scheduling

- The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization.
- The objective of time sharing is to switch the CPU among processes so frequently that users can interact with each program while it is running.
- To meet these objectives, the process scheduler selects an available process (possibly from a set of several available processes) for program execution on the CPU.
  - For a single-processor system, there will never be more than one running process.
  - If there are more processes, the rest will have to wait until the CPU is free and can be rescheduled.





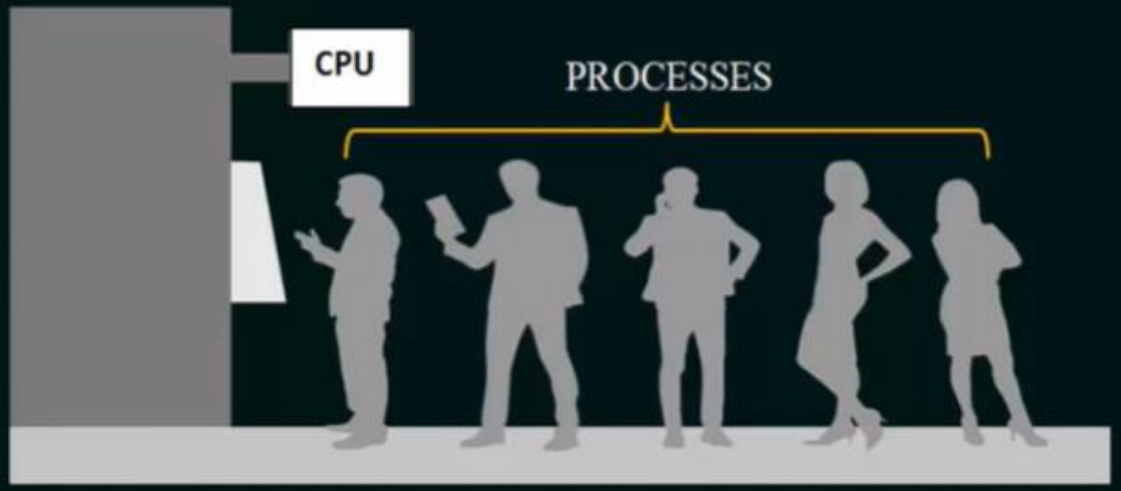
Scheduling Queues

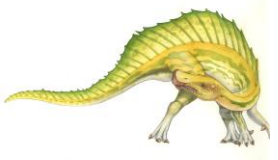
JOB QUEUE

As processes enter the system, they are put into a **job queue**, which consists of all processes in the system.

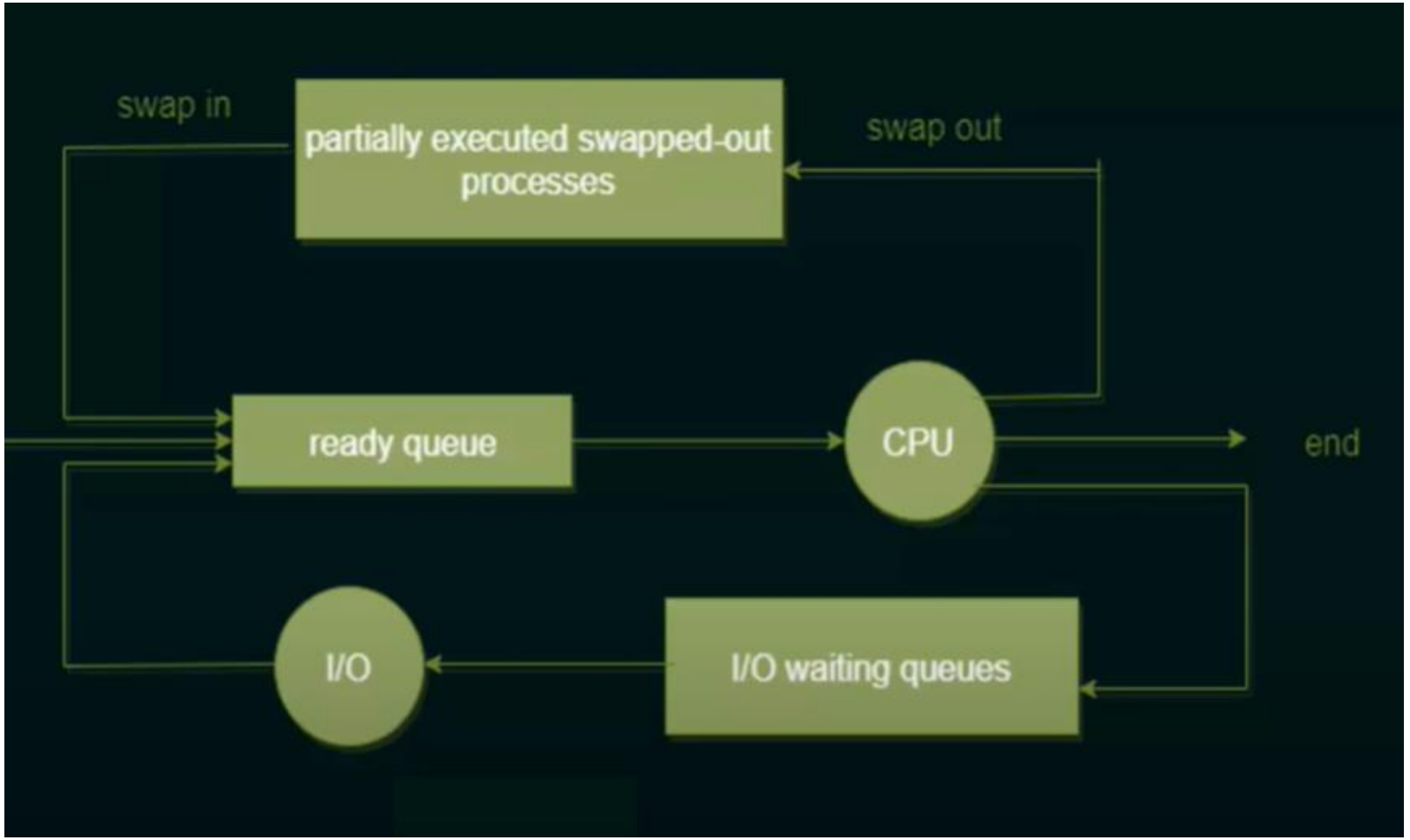
READY QUEUE

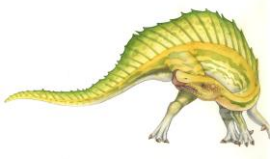
The processes that are residing in main memory and are ready and waiting to execute are kept on a list called the **ready queue.**







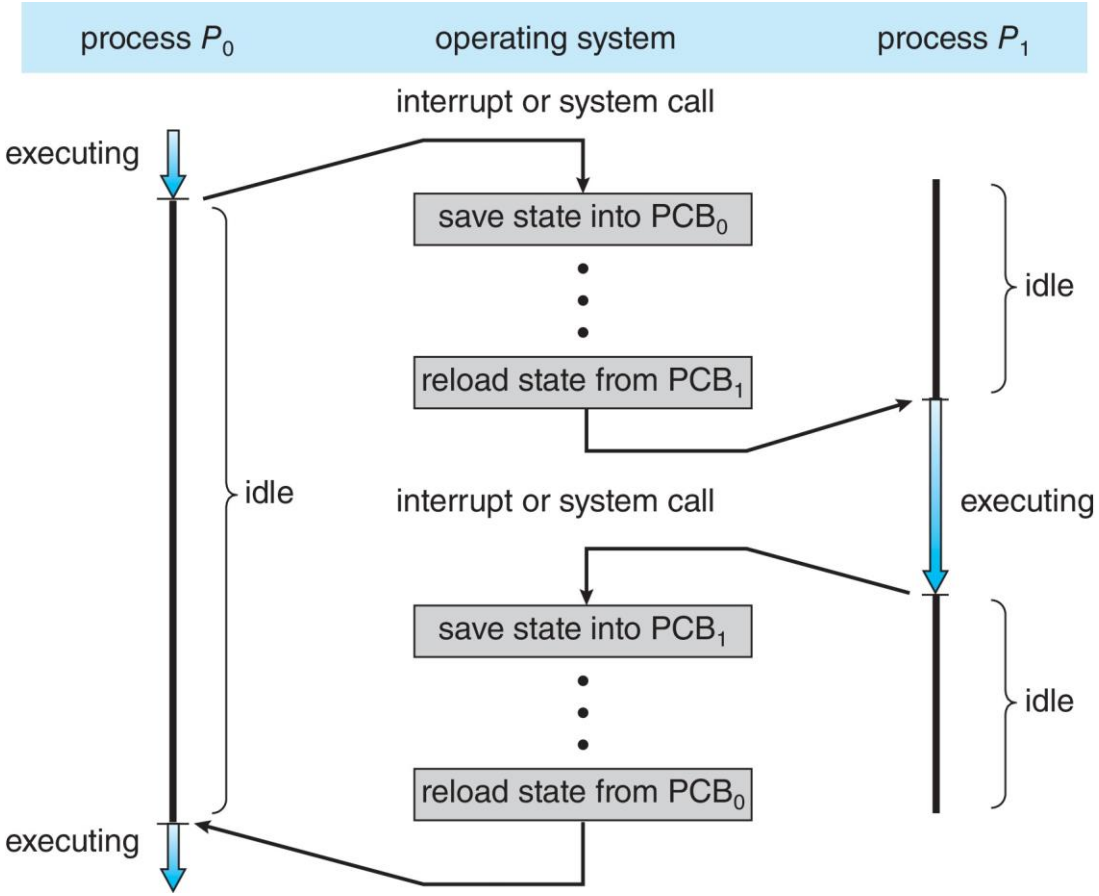

swap in
partially executed swapped-out processes
swap out
ready queue
CPU
end
I/O
I/O waiting queues





CPU Switch From Process to Process

A **context switch** occurs when the CPU switches from one process to another.

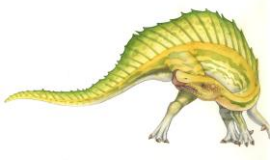






Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is pure overhead (cost: resources, time); the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once

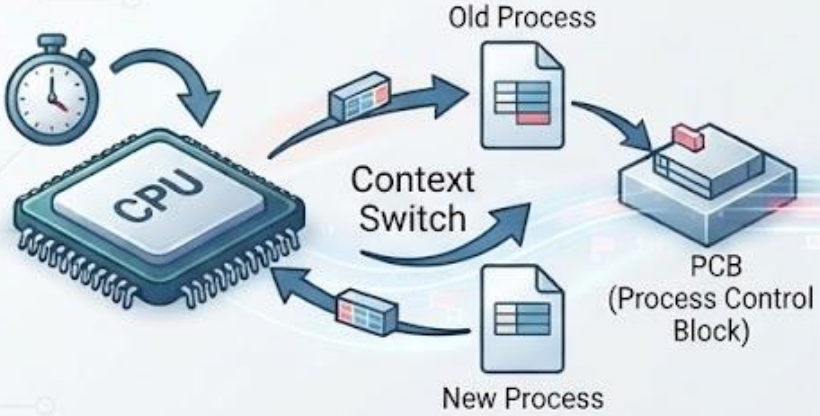




Context Switch

Understanding Process Context Switching

1. Context Switch.



2. Context in PCB.



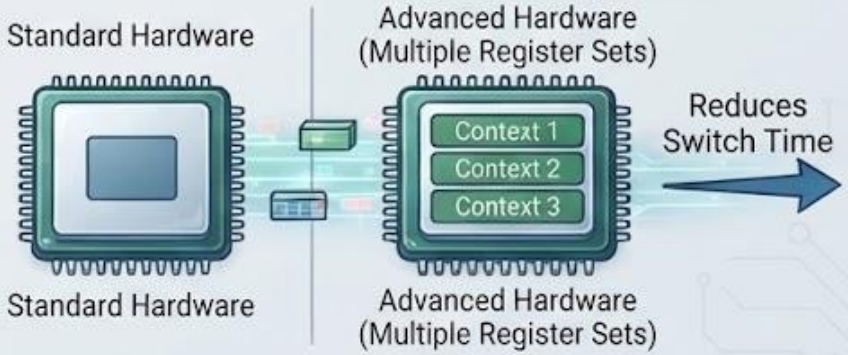
3. Overhead



Cost: Resources, Time.
System does no useful work while switching.

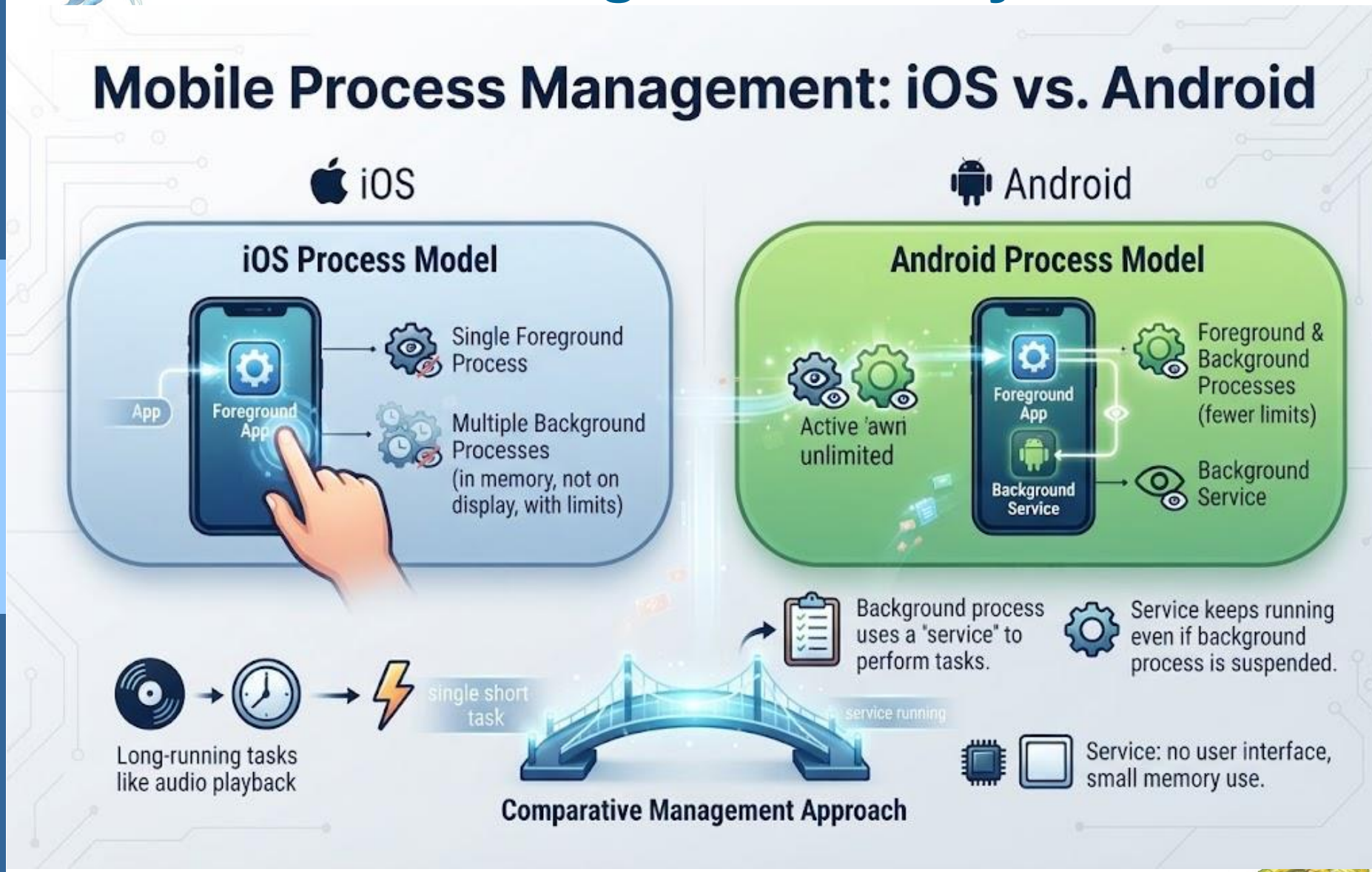


4. Hardware Support



Operating System Concepts - Process Management

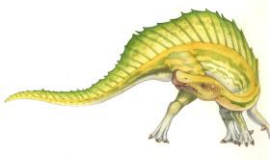
Multitasking in Mobile Systems





Multitasking in Mobile Systems

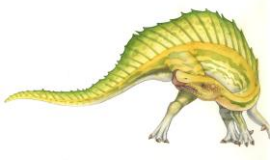
- Some mobile systems (e.g., early version of iOS) allow only one process to run, others suspended
- Due to screen real estate, user interface limits iOS provides for a
  - Single **foreground** process- controlled via user interface
  - Multiple **background** processes– in memory, running, but not on the display, and with limits
  - Limits include single, short task, receiving notification of events, specific long-running tasks like audio playback
- Android runs foreground and background, with fewer limits
  - Background process uses a **service** to perform tasks
  - Service can keep running even if background process is suspended
  - Service has no user interface, small memory use



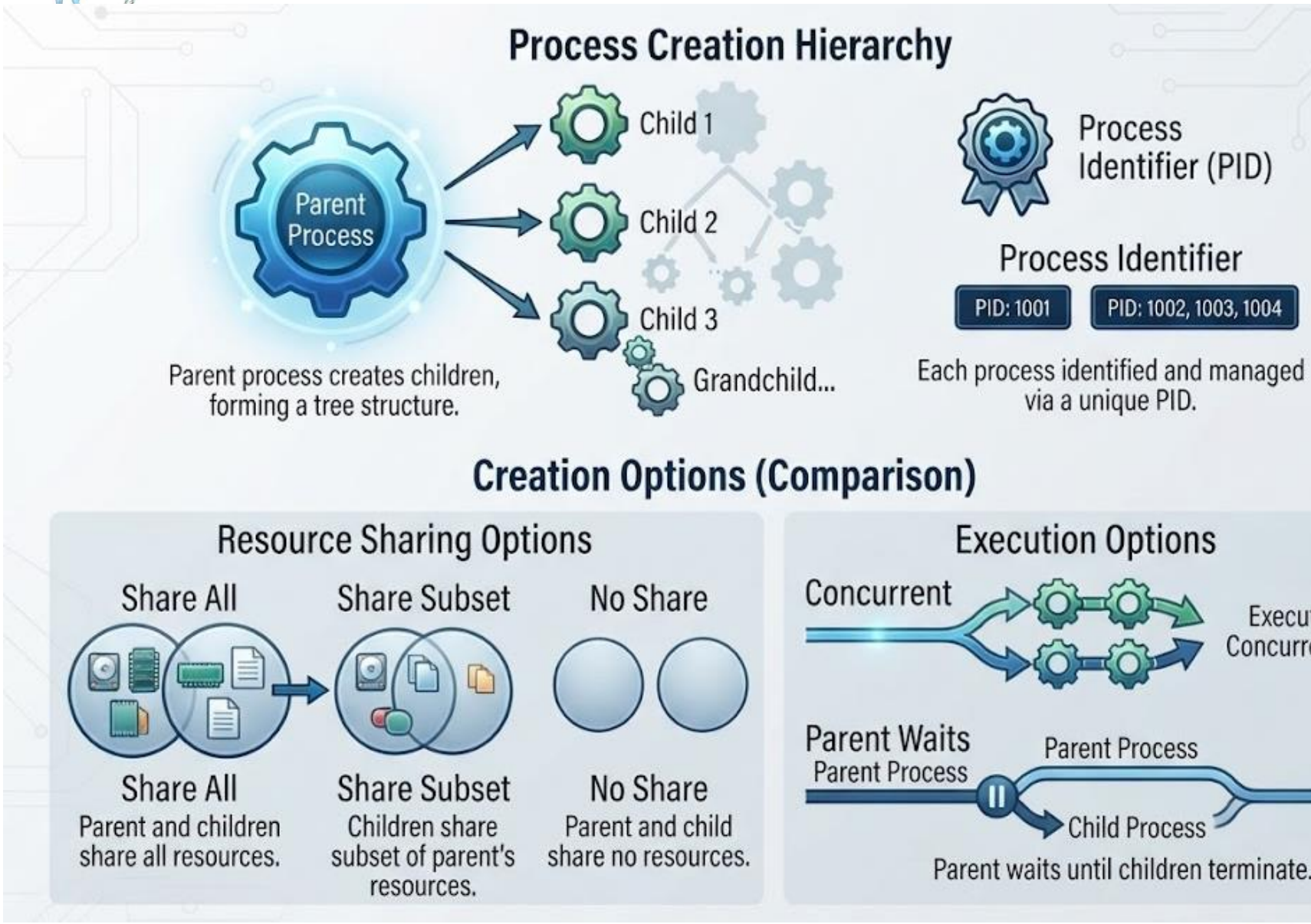


Operations on Processes

- System must provide mechanisms for:
  - Process creation
  - Process termination



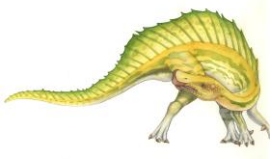
Process Creation





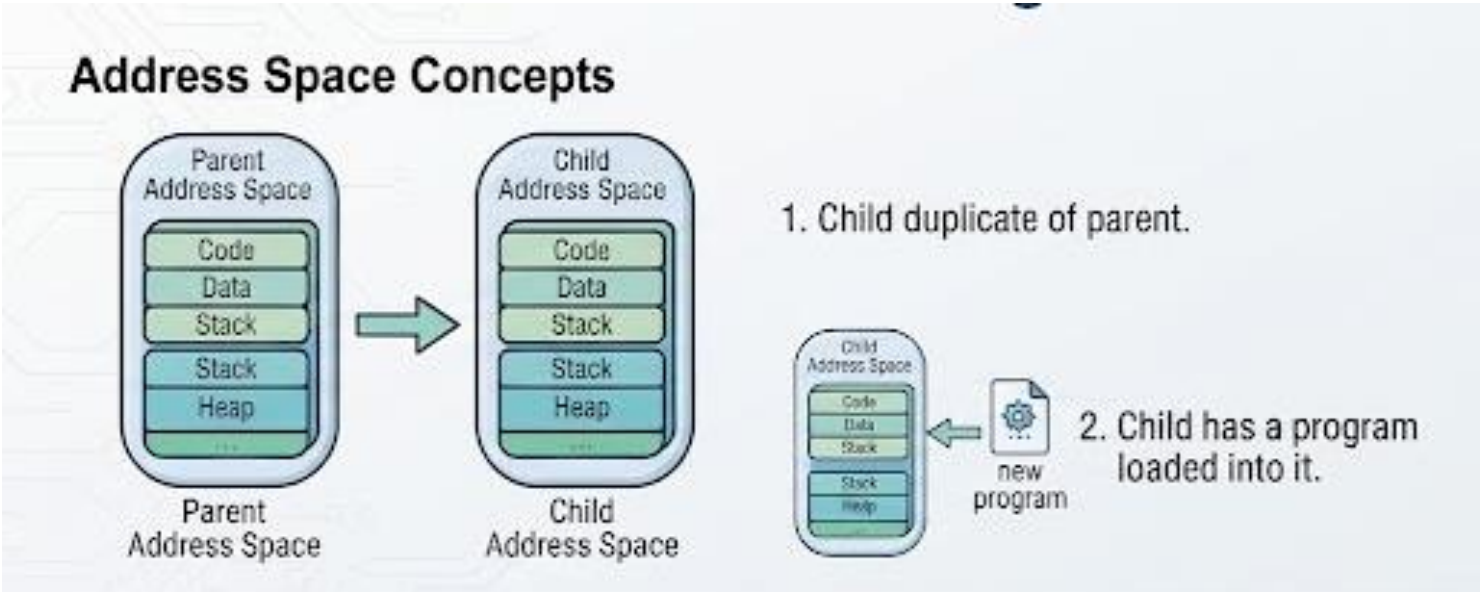
Process Creation

- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier** (**pid**)
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate

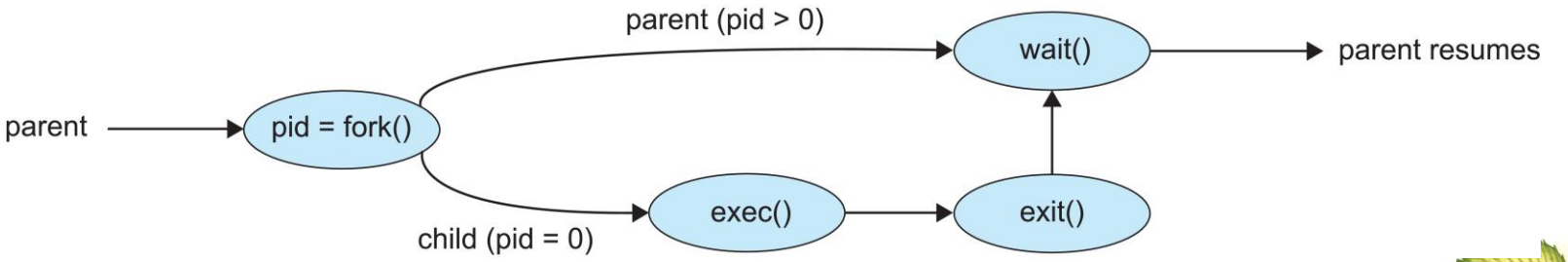




Process Creation (Cont.)



- UNIX examples

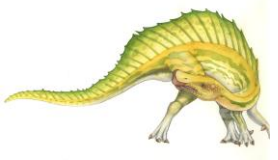






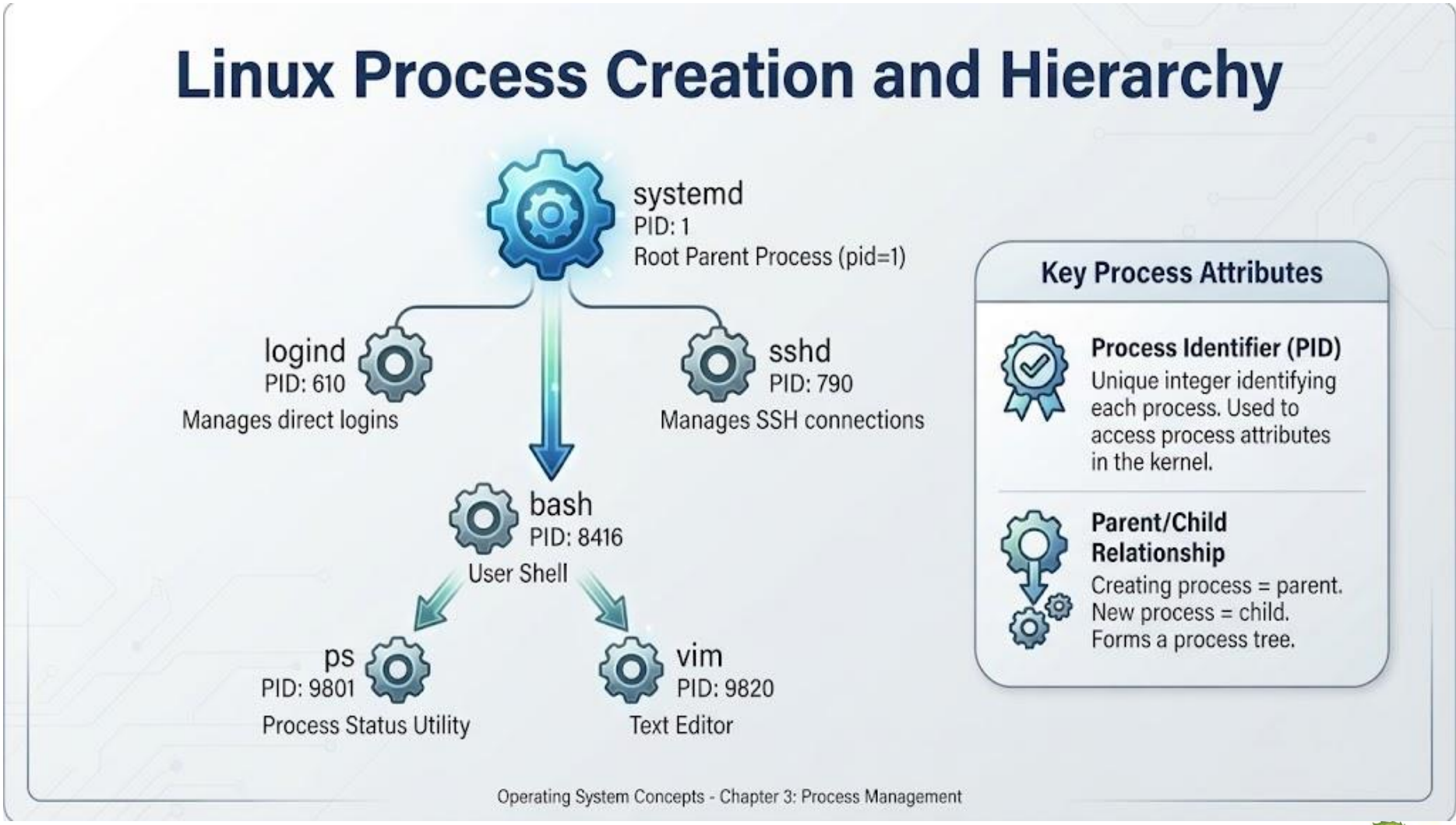
Process Creation (Cont.)

- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - **`fork()`** system call creates new process
  - **`exec()`** system call used after a **`fork()`** to replace the process' memory space with a new program
  - Parent process calls **`wait()`** waiting for the child to terminate





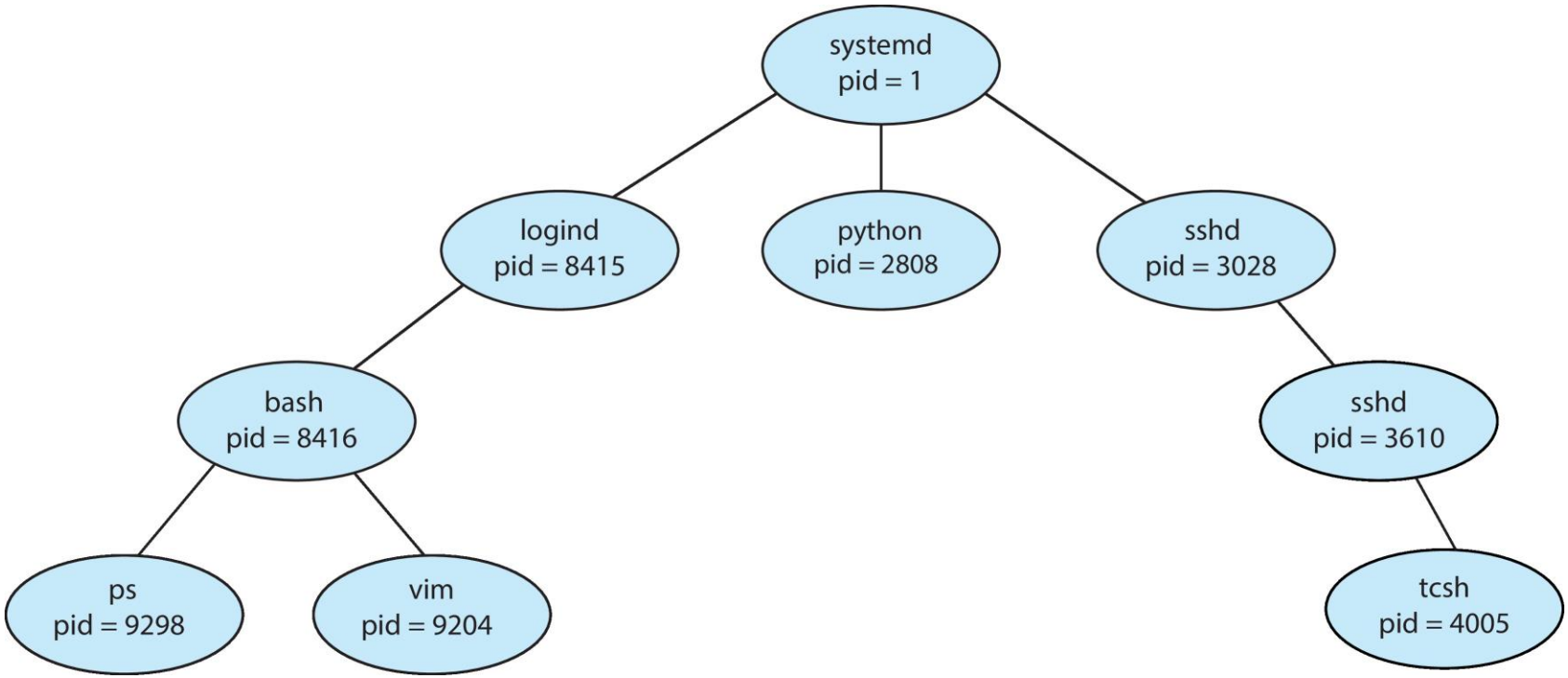
A Tree of Processes in Linux

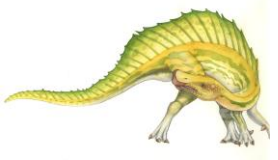






A Tree of Processes in Linux







C Program Forking Separate Process

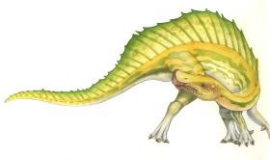
```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```





Creating a Separate Process via Windows API

```
#include <stdio.h>
#include <windows.h>

int main(VOID)
{
STARTUPINFO si;
PROCESS_INFORMATION pi;

    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
     "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
     NULL, /* don't inherit process handle */
     NULL, /* don't inherit thread handle */
     FALSE, /* disable handle inheritance */
     0, /* no creation flags */
     NULL, /* use parent's environment block */
     NULL, /* use parent's existing directory */
     &si,
     &pi))
    {
      fprintf(stderr, "Create Process Failed");
      return -1;
    }
    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");

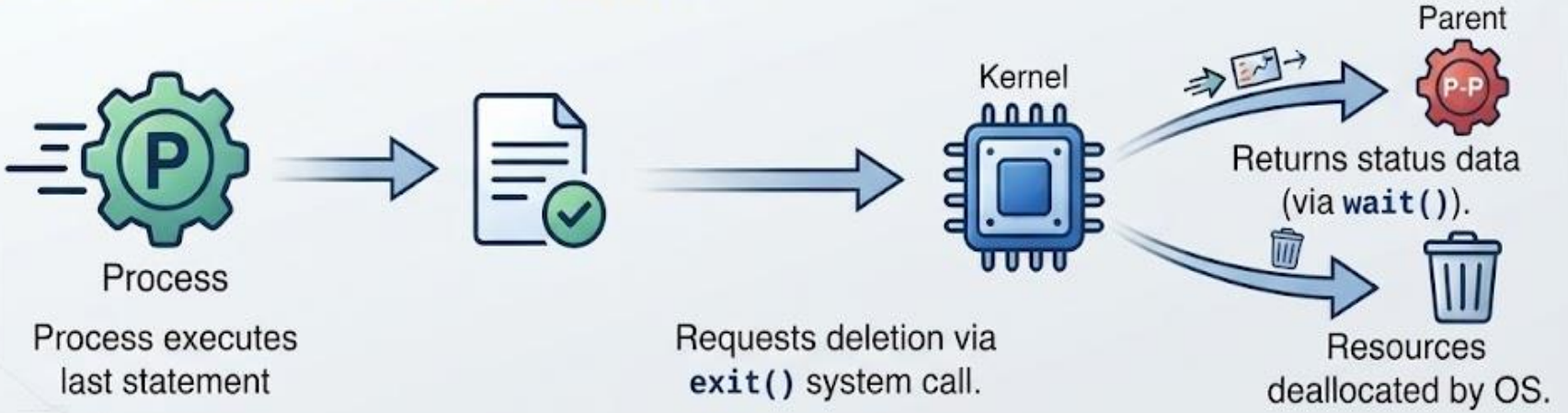
    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```



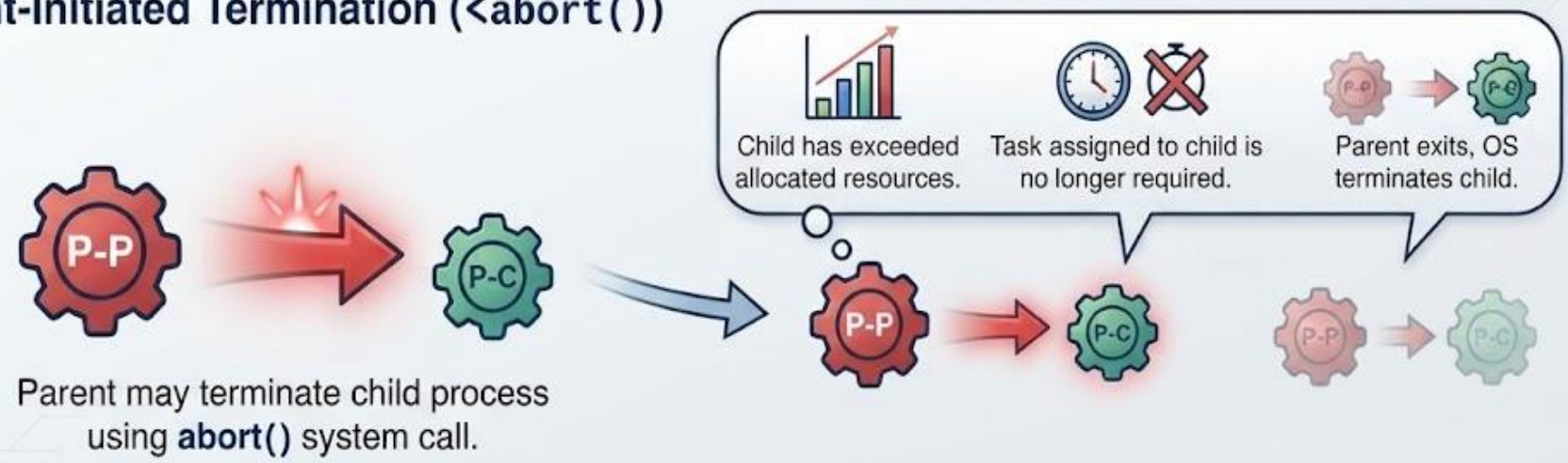


Process Termination

Normal Process Termination (`exit()`)



Parent-Initiated Termination (`<abort()`)





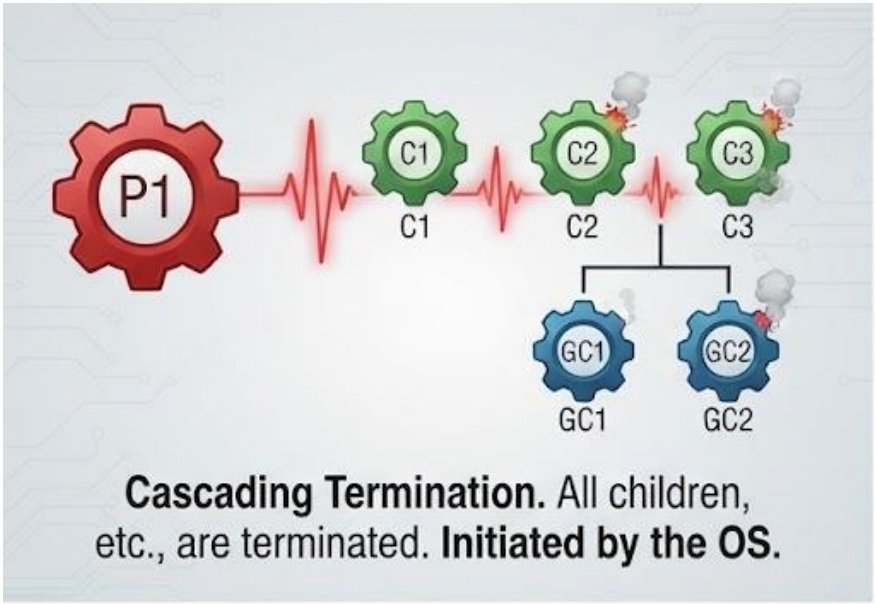
Process Termination

- Process executes last statement and then asks the operating system to delete it using the **`exit()`** system call.
  - Returns status data from child to parent (via **`wait()`**)
  - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **`abort()`** system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting, and the operating systems does not allow a child to continue if its parent terminates

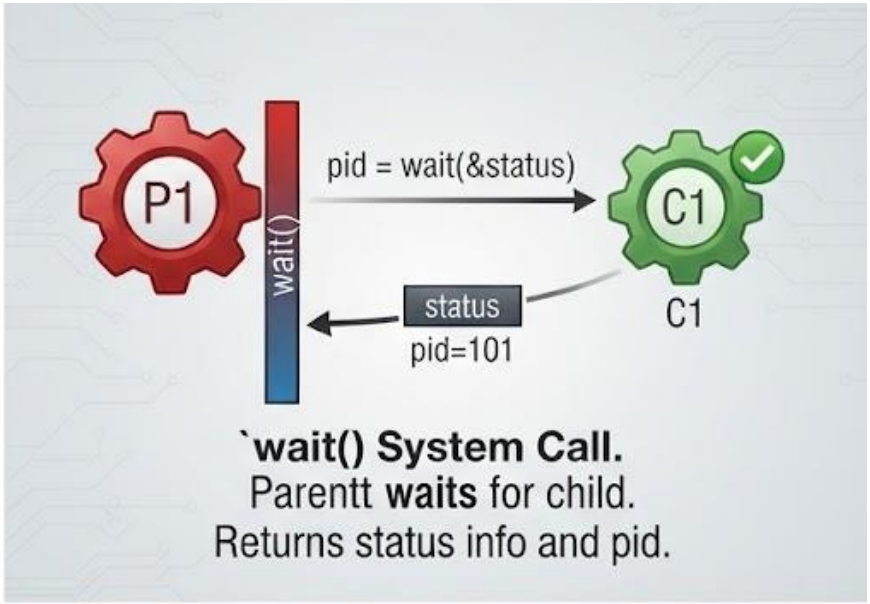




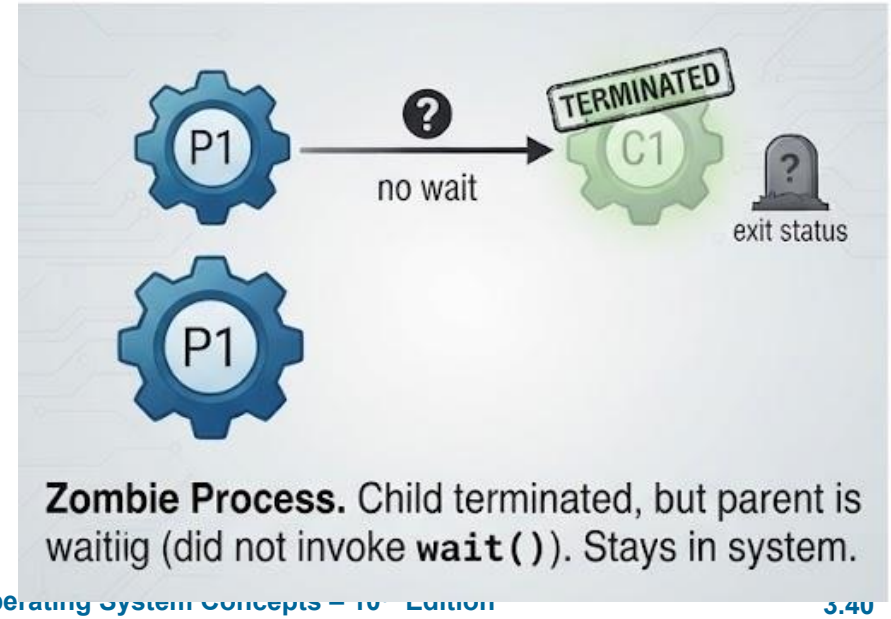
Process Termination



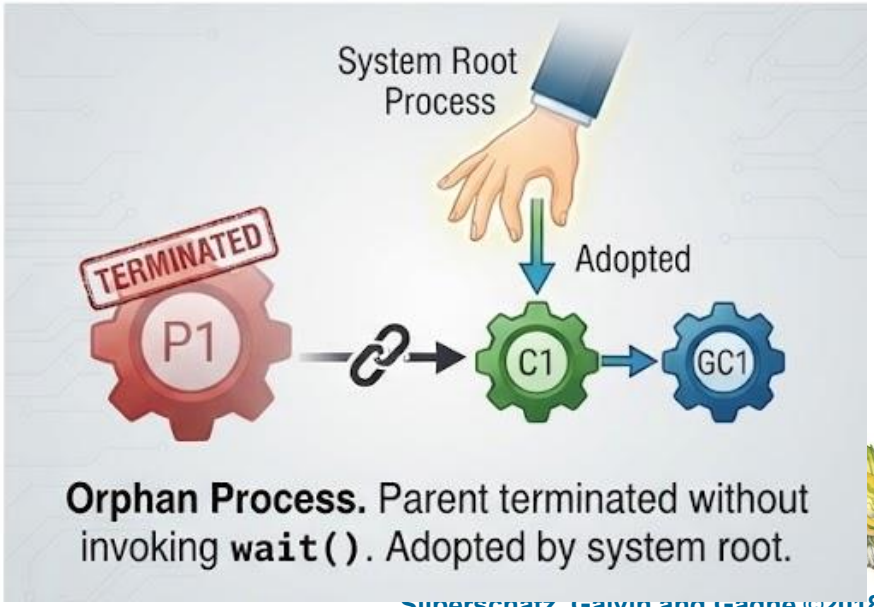
Cascading Termination. All children, etc., are terminated. **Initiated by the OS.**



`wait() System Call.
Parentt **waits** for child.
Returns status info and pid.



Zombie Process. Child terminated, but parent is waitiig (did not invoke **`wait()`**). Stays in system.



Orphan Process. Parent terminated without invoking **`wait()`**. Adopted by system root.



Process Termination

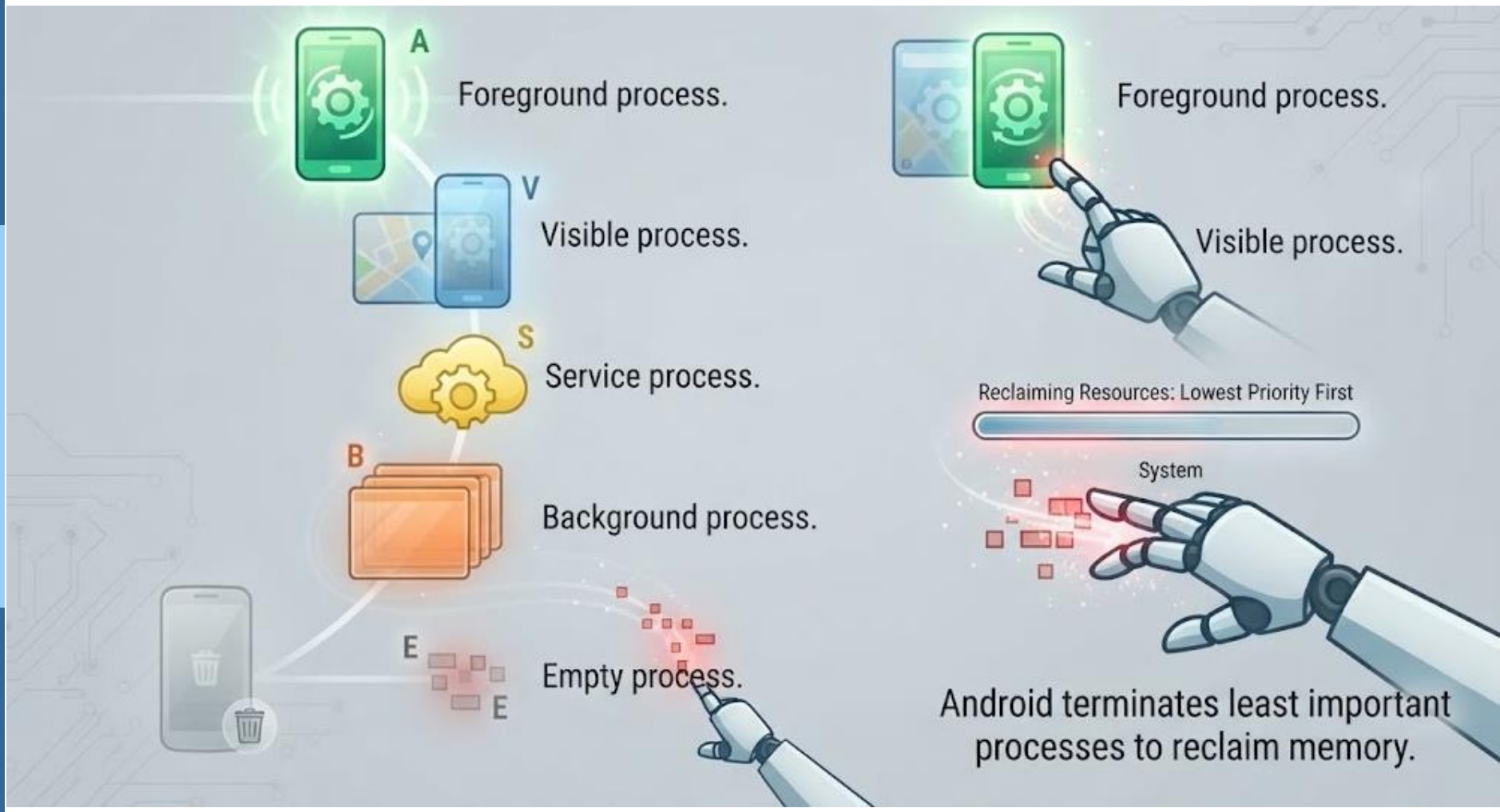
- Some operating systems do not allow child to exists if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading termination.** All children, grandchildren, etc., are terminated.
  - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **`wait()`** system call. The call returns status information and the pid of the terminated process

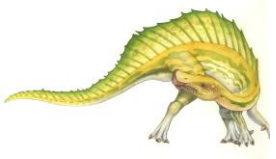
```
pid = wait(&status);
```

- If no parent waiting (did not invoke **`wait()`**) process is a **zombie**
- If parent terminated without invoking **`wait()`**, process is an **orphan**



Android Process Importance Hierarchy

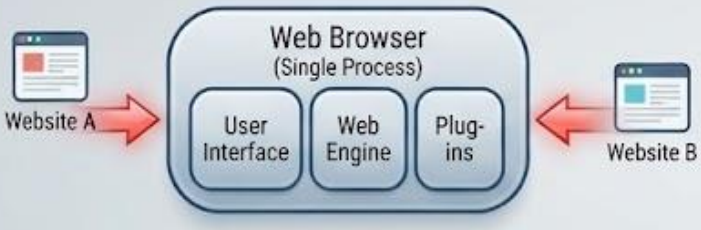






Multiprocess Architecture – Chrome Browser

Single-Process Browser Model (Legacy)

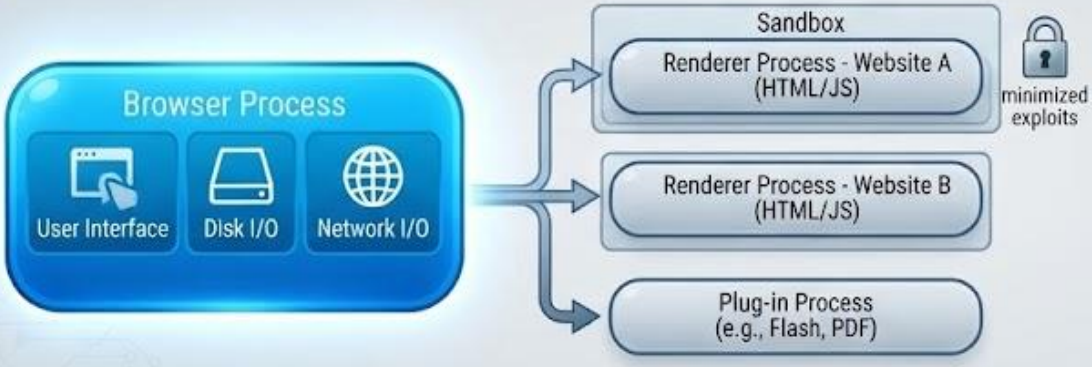


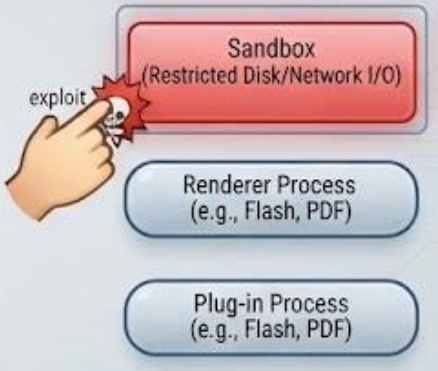
Many web browsers ran as single process (some still do).



If one website causes trouble, entire browser can hang or crash.

Google Chrome Multi-Process Model (Modern)





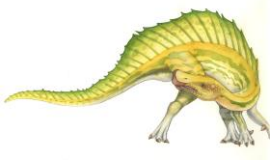
Google Chrome Browser is multiprocess with 3 different types of processes: Browser, Renderer (new for each websitte, runs in sandbox), Plug-in.





Multiprocess Architecture – Chrome Browser

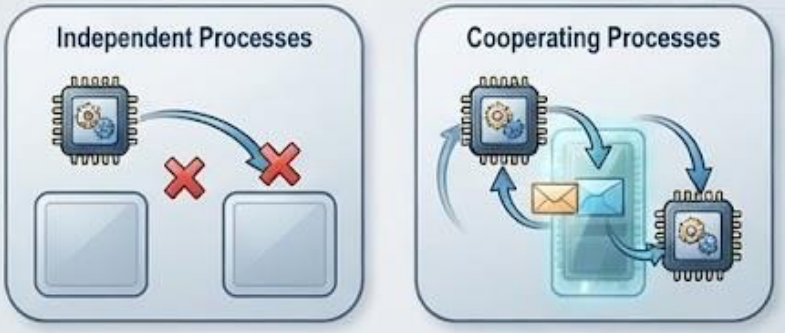
- Many web browsers ran as single process (some still do)
  - If one web site causes trouble, entire browser can hang or crash
- Google Chrome Browser is multiprocess with 3 different types of processes:
  - **Browser** process manages user interface, disk and network I/O
  - **Renderer** process renders web pages, deals with HTML, Javascript. A new renderer created for each website opened
    - Runs in **sandbox** restricting disk and network I/O, minimizing effect of security exploits
  - **Plug-in** process for each type of plug-in





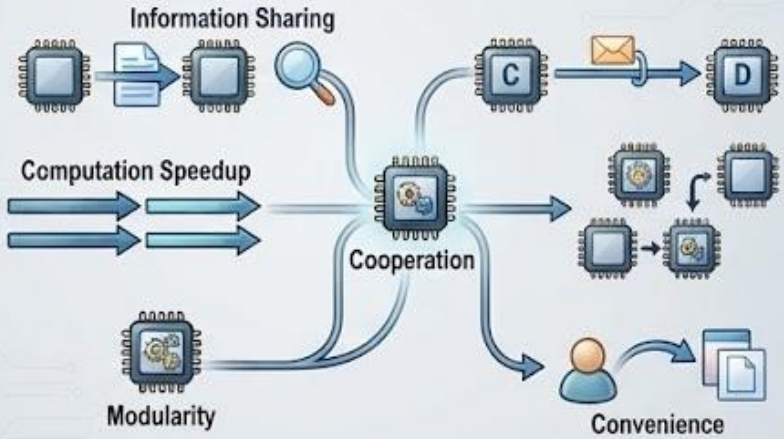
Interprocess Communication

Independent vs. Cooperating Processes



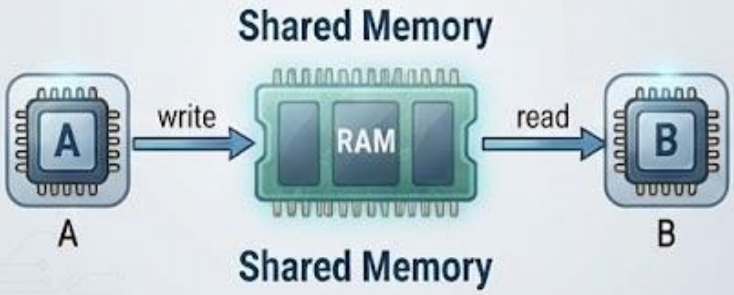
Processes within a system may be **independent** or **cooperating**. Cooperating processes can affect or be affected by other processes, including sharing data.

Reasons for Cooperating Processes



IPC Models - Shared Memory vs. Message Passing

Cooperating processes need **interprocess communication (IPC)**. Two models of IPC.



IPC Models - Shared Memory vs. Message Passing

Cooperating processes need **interprocess communication (IPC)**. Two models of IPC.







Interprocess Communication

- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**

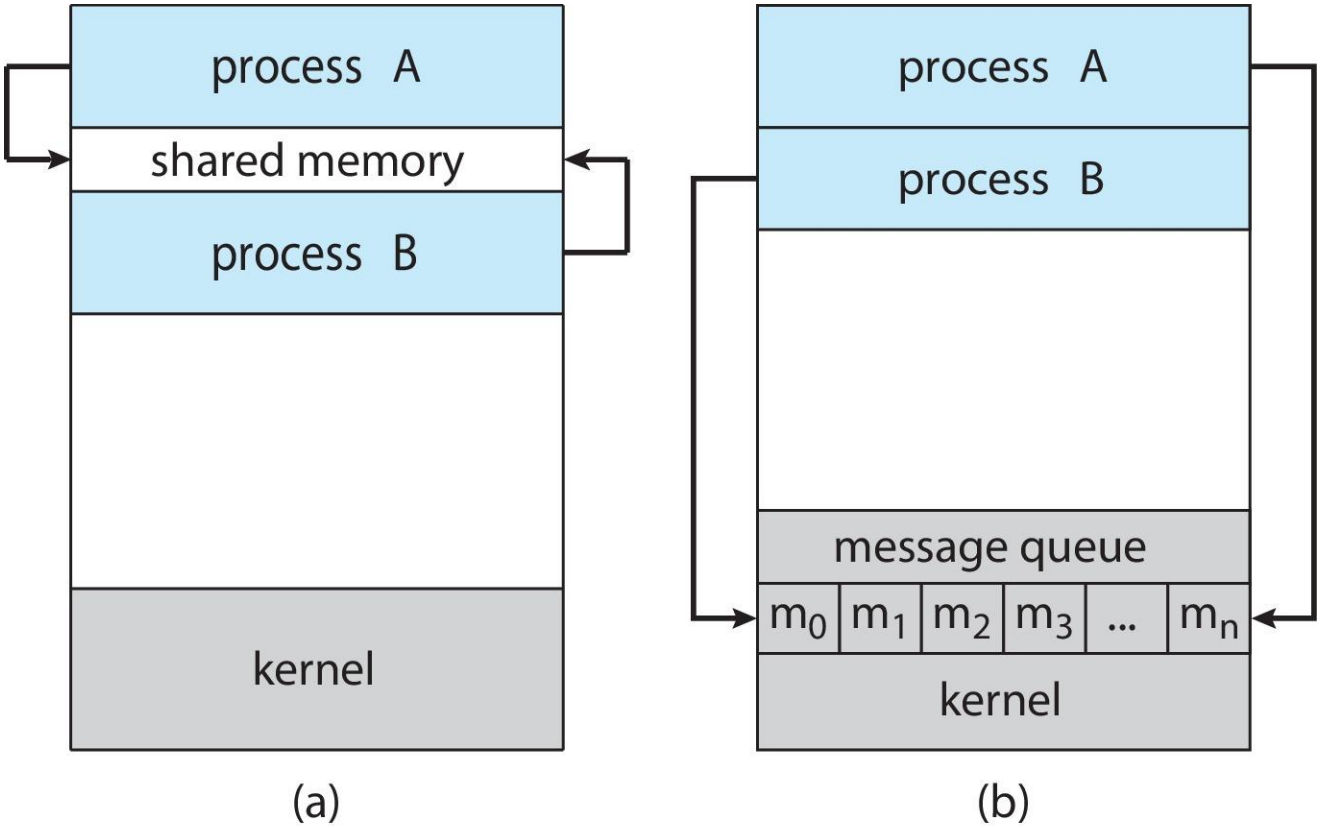




Communications Models

(a) Shared memory.

(b) Message passing.







Producer-Consumer Problem

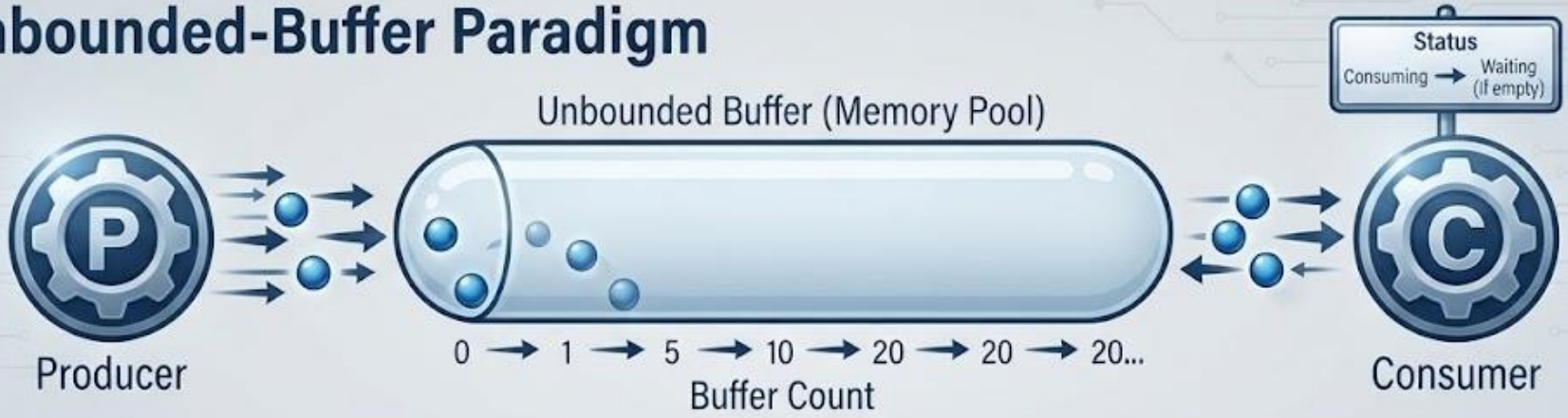
- Paradigm for cooperating processes:
  - *producer* process produces information that is consumed by a *consumer* process
- Two variations:
  - **unbounded-buffer** places no practical limit on the size of the buffer:
    - Producer never waits
    - Consumer waits if there is no buffer to consume
  - **bounded-buffer** assumes that there is a fixed buffer size
    - Producer must wait if all buffers are full
    - Consumer waits if there is no buffer to consume





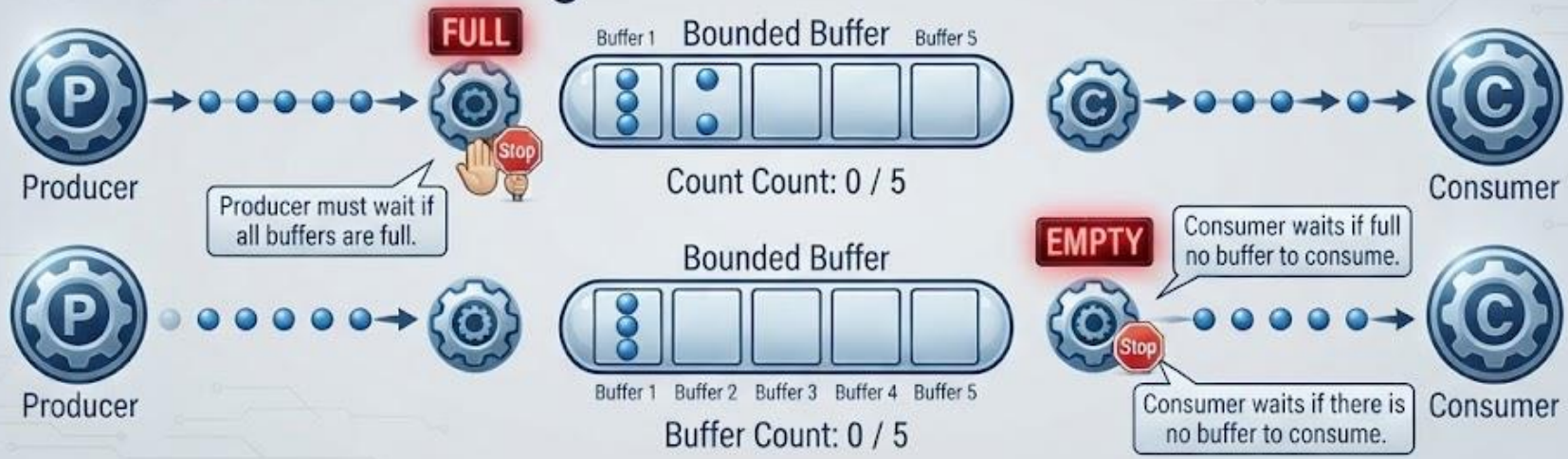
Producer-Consumer Problem

Unbounded-Buffer Paradigm



unbounded-buffer places **no practical limit on the size of the buffer:**
Producer never waits. Consumer waits if there is no buffer to consume.

Bounded-Buffer Paradigm





Inter process Communication (IPC) – Shared Memory

Shared Memory Overview

An area of memory shared among the processes that wish to communicate.



User Process Control

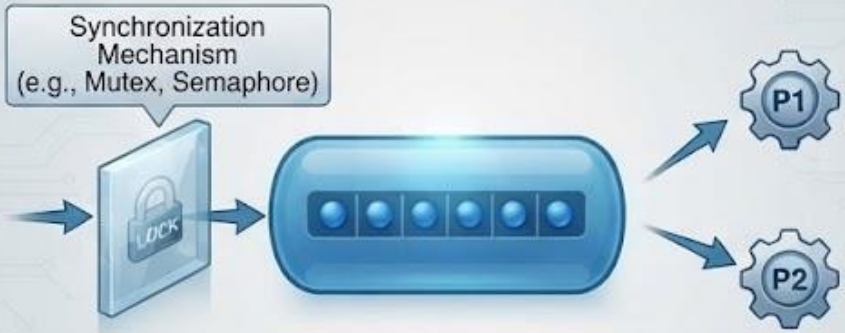


Synchronization Issue



Major issues is to provide mechanism that will allow user processes to **synchronize** their actions when they access shared memory.

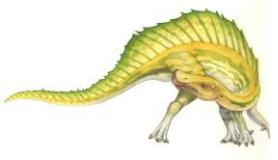
Synchronized Order





IPC – Shared Memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.





Bounded-Buffer

IPC - Shared Memory: Bounded Buffer Mechanism

Variables in Shared Memory

Shared Variables Definition

```
#define BUFFER SIZE 10
typedef struct {
   . . .
} item;

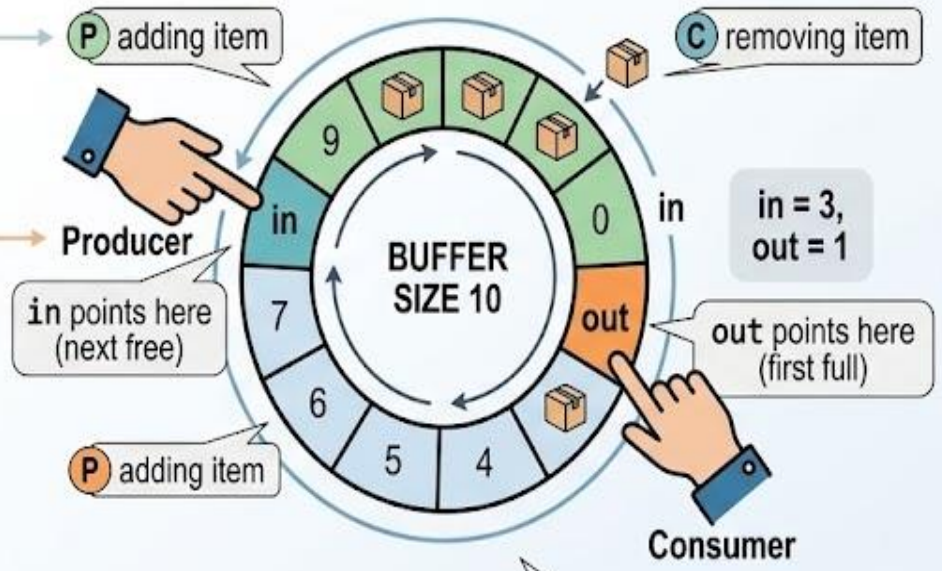
item buffer[BUFFER SIZE];
int in = 0;
int out = 0;
```

buffer is the shared item array

in points to next free slot

out points to first full slot

The Circular Array Buffer



Buffer State Conditions



Buffer is **Empty**
in == out



Buffer is **Full**
((in + 1) % BUFFER SIZE) == out

Modulo operator (%) enables **wrap-around** (circular logic)





Process – Shared Memory

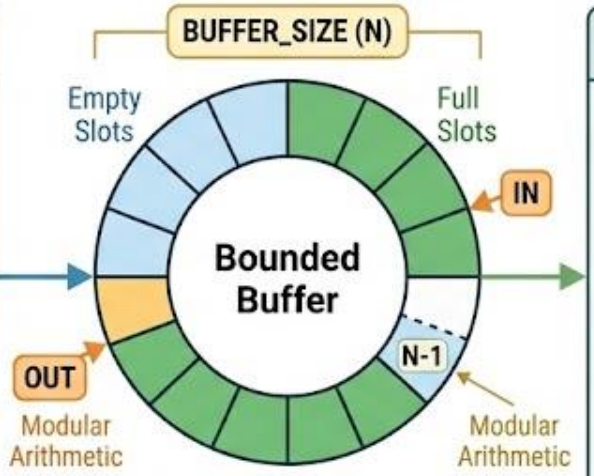
Bounded Buffer Capacity & Complete Algorithms

When he BUFFER_SIZE limit is **minus 1 (N-1)**

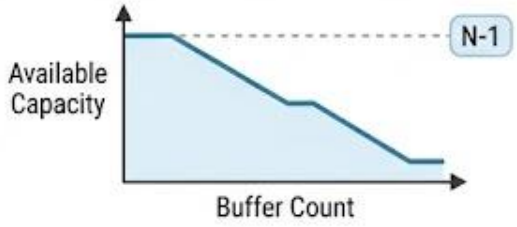
PRODUCER ALGORITHM (Complete)

```
// Producer Code - Correct 'N-1' limit
void producer() {
  while (true) {
    // 1. Check if buffer is full (N-1)
    if ( (in + 1) % BUFFER_SIZE == out ) {
      // Buffer full, wait until a slot is free
      // wait_for_consumer();
    } else {
      // 2. Add an item
      buffer[in] = item;

      // 3. Update 'IN' pointer, wrapping around
      in = (in + 1) % BUFFER_SIZE;
    }
  }
}
```



CAPACITY CONDITION:
Max Full Slots = BUFFER_SIZE - 1.
A dedicated slot remains 'Empty' to distinguish 'Full' and 'Empty' states using IN and OUT pointers.



CONSUMER ALGORITHM (Complete)

```
// Consumer Code - Consuming available items
void consumer() {
  while (true) {
    // 1. Check if buffer is empty
    if ( in == out ) {
      // Buffer empty, wait for items
      // wait_for_producer();
    } else {
      // 2. Consume an item
      item = buffer[out];

      // 3. Update 'OUT' pointer, wrapping around
      out = (out + 1) % BUFFER_SIZE;
    }
  }
}
```





What about Filling all the Buffers?

Shared Counter for Producer-Consumer Bounded-Buffer Solution

The Solution (Conceptual)



SHARED COUNTER

- **Problem:** Fill all buffers safely.
- Shared integer counter tracks **full buffers.**
- **Initially**, counter is 0.





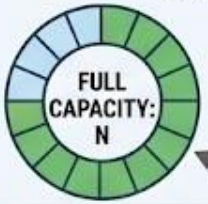
PRODUCER

PRODUCER: Increments counter after each new item produced.

CONSUMER: Decrements counter after each item consumed.



CONSUMER



vs. A dedicaed slot remains distinguish 'Full' and 'Empty' state in OUT pointers.

Max Full Slots = **N-1**

Producer-Consumer Synchronization (Naive Counter)

CRITICAL CONCURRENCY PROBLEM!



RACE CONDITION
CRITICAL CONCURRENCY PROBLEM!

Wait... This Naive This Naive Counter Needs Locking!
(Mutex, Semaphores...)

```
// Producer Loop
while (true) {
    // ... produce item ...
    while (counter == BUFFER_SIZE) {
        // Buffer full, wait...
    }
    // ... add item to buffer ...
    counter++; // naive increment
}

// Consumer Loop
while (true) {
    while (counter == 0) {
        // Buffer empty, wait...
    }
    // ... remove item ...
    counter--; // naive decrement
    // ... consume item ...
}
```

RACE CONDITION RISK

RACE CONDITION RISK

While conceptually sound to track all slots, **a simple un-synchronized counter** introduces **severe concurrency errors** (race conditions). This is a conceptual view before proper synchronization.





What about Filling all the Buffers?

- Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers.
- We can do so by having an integer **`counter`** that keeps track of the number of full buffers.
- Initially, **`counter`** is set to 0.
- The integer **`counter`** is incremented by the producer after it produces a new buffer.
- The integer **`counter`** is and is decremented by the consumer after it consumes a buffer.





Producer

```
while (true) {
        /* produce an item in next produced */

        while (counter == BUFFER_SIZE)
                ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
        counter++;
}
```





Consumer

```
while (true) {
        while (counter == 0)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
         counter--;
        /* consume the item in next consumed */
}
```





What about Filling all the Buffers?

Shared Counter for Producer-Consumer Bounded-Buffer Solution

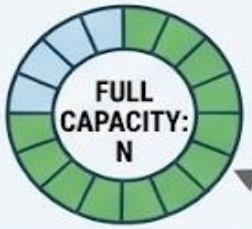
The Solution (Conceptual)



SHARED COUNTER

- **Problem:** Fill all buffers safely.
- Shared integer counter tracks **full tacks buffers**.
- **Initially**, counter is 0.





VS. A dedicaed slot remains distinguish 'Full' and 'Empty' state in OUT pointers.

Max Full Slots = **N-1**

CRITICAL CONCURRENCY PROBLEM!



RACE CONDITION CRITICAL CONCURRENCY PROBLEM!

IMPORTANT NOTE: Corrected Race Condition with a MUTEX

```
// Corrected Producer Loop (with Mutex)
while (true) {
  // ... produce item ...
  acquire(counter_mutex); // Lock acquired
  // Wait condition checked *inside* mutex
  while (counter == BUFFER_SIZE) {
    release(counter_mutex); // Release &
                re-acquire to prevent deadlock
    // busy wait or condition variable
    acquire(counter_mutex);
  }
  // ... add item to buffer ...
  counter++; // SAFE INCREMENT inside lock
  release(counter_mutex); // Lock released
}
```

wait condition re-evaluation



A **Mutex** (or other synchronization primitive) ensures atomic operations on shared variables.

While conceptually sound to track all slots, **a simple un-synchronized counter** introduces **severe concurrency errors** (race condition). This is a conceptual view before proper synclonization.

**This slide shows the naive counter and an important correction note.

IPC RACE CONDITIONS:
Definitions, Examples, and Solutions

What is a Race Condition? A race condition occurs when two or more processes concurrently access and modify shared data, and the final result depends on the order or timing of their execution.

Process P1

Suppose P1 and P2 want to increment a shared variable **count = 5**. (Initial: 5)

Process P2

STEP-BY-STEP RACE CONDITION EXECUTION



Actual result: **6** our lost increment **vs.** **Expected result: 7** proper increment

Both processes read the same original value before either update was completed. **ONE INCREMENT IS LOST.**

How can a race condition be prevented?

Mutex locks: Allow only one process at a time to access the critical section.

Semaphores: Control access to shared resources.

Atomic operations: Perform an operation indivisibly.

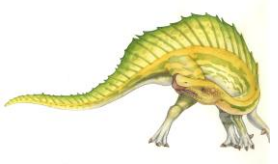
Monitors: Coordinate access to shared data through synchronized procedures.

Key exam point: A race condition occurs when the outcome depends on the timing or interleaving of concurrent processes accessing shared resources.



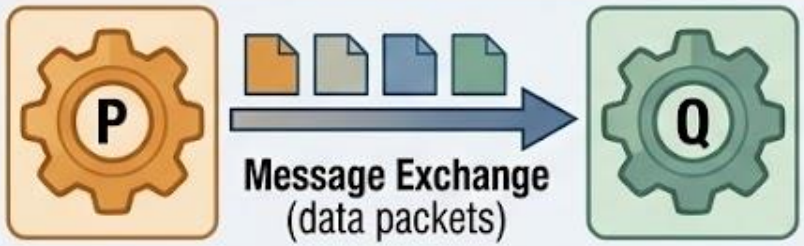
Race Condition (Cont.)

- Question – why was there no race condition in the first solution (where at most N – 1) buffers can be filled?
- More in Chapter 6.



IPC: MESSAGE-PASSING FACILITY

The Concept: Communicate Without Shared Memory



Provides a mechanism for cooperating processes to communicate and synchronize **without** sharing the same address space. Particularly useful in a distributed environment (different computers over network).

Example: Internet Chat Program.



Core Message-Passing Operations

`send(message)` — Transmit a message

`receive(message)` — Receive a message

Message Size Tradeoff: A Common OS Design Principle

	Fixed-Sized Messages		Variable-Sized Messages
System Implementation	**Straightforward**	vs.	**Complex**
Programming Task	**Difficult** (manual chunking)	vs.	**Simpler** (natural data)

Design Tradeoff: Simplicity for Programmer vs. Implementation Complexity

Establishing a Communication Link (Logical Perspective)

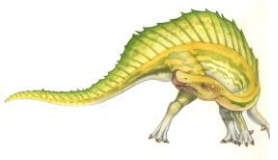
- **Direct or Indirect Communication** — Logical Communication Link — Naming processes explicitly vs. using intermediary mailboxes/ports
- **Synchronous or Asynchronous Communication** — Sender waits/blocks vs. sender continues (buffered)
- **Automatic or Explicit Buffering** — OS manages queue vs. explicit buffer size management

→ Issues related to each feature are explored next.



IPC – Message Passing

- Processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **`send`**(*message*)
  - **`receive`**(*message*)
- The *message* size is either fixed or variable



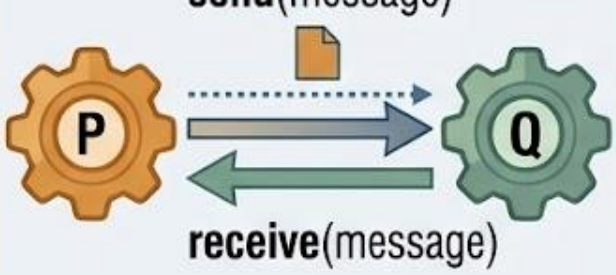
IPC: Communication Link Requirements and Design Issues

Communication Steps for Processes P & Q

1. Establish a communication link between P and Q.



2. Exchange messages via Send/Receive.



IPC Communication Link Design & Implementation Issues

1. How are links established?
Logical (naming) and Physical (channel) creation methods.

2. Can a link be associated with > 2 processes?
Point-to-point (2) or Multi-point (n) communication.

3. How many links between each pair?
Single, dual, or multi-link architectures.

4. What is the capacity of a link?
Buffering: Zero, Bounded (fixed), or Unbounded capacity.

5. Fixed or variable message size?
Fixed-sized vs. Variable-sized messages.

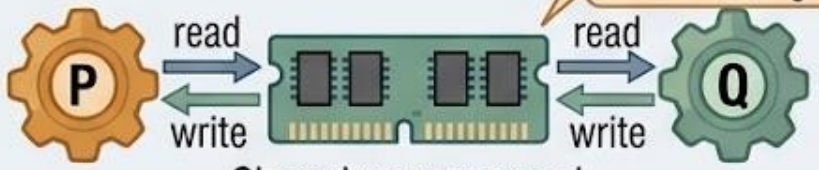
6. Is a link unidirectional or bi-directional?
One-way data flow vs. Two-way simultaneous data flow.

IPC: Communication Link Implementation

Physical Implementation

1. Shared Memory

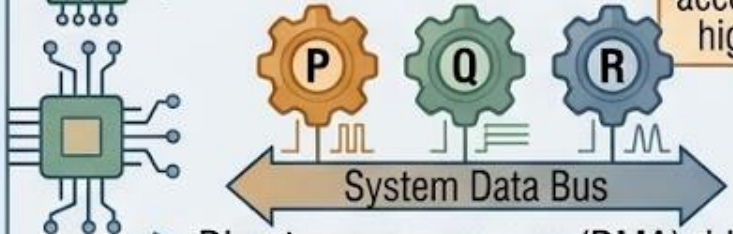
Fastest, kernel-bypassed data exchange.



Shared memory pool

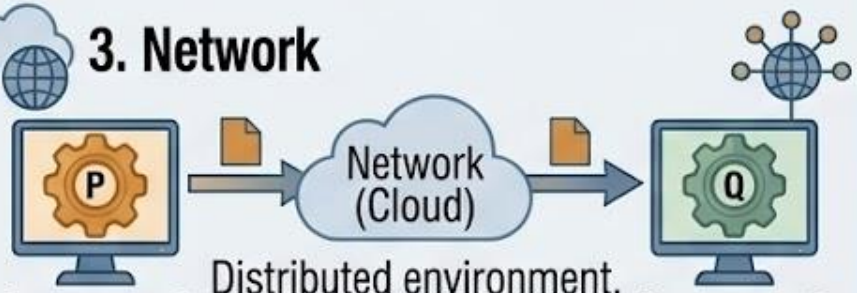
2. Hardware Bus

Direct memory access (DMA), high-speed.



Direct memory access (DMA), high-speed.

3. Network



Computer A

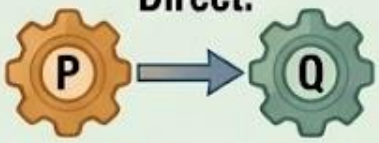
Distributed environment, uses network stack.

Computer B

Logical Implementation

1. Direct or Indirect

Direct:



P calls Q specifically

Indirect:



Uses intermediate shared object

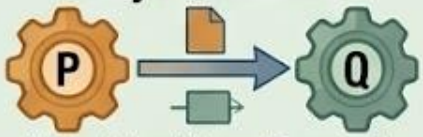
2. Synchronous or Asynchronous

Synchronous



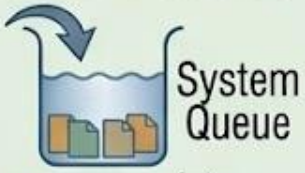
Both block until transfer is done.

Asynchronous

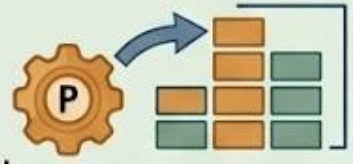


Non-blocking, P sends data and proceeds.

3. Automatic or Explicit Buffering



OS manages data buffer automatically



User process manages buffer explicitly.

IPC: DIRECT COMMUNICATION (Naming Processes Explicitly)

THE CONCEPT: EXPLICIT NAMING



Process must name the *recipient* directly.

```
send(P, message)
```

– *send* a message to process *P*



Process must name the *sender* directly.

```
receive(Q, message)
```

– *receive* a message from process *Q*



PROPERTIES OF THE COMMUNICATION LINK



1. Links are established automatically.

Simply by issuing a send or receive operation, the link is created without user intervention.



2. A link is associated with exactly one pair of communicating processes.

No multi-point links; strictly point-to-point.



3. Between each pair there exists exactly one link.

Multiple links between the single
Multiple links between the *same* pair are not allowed.

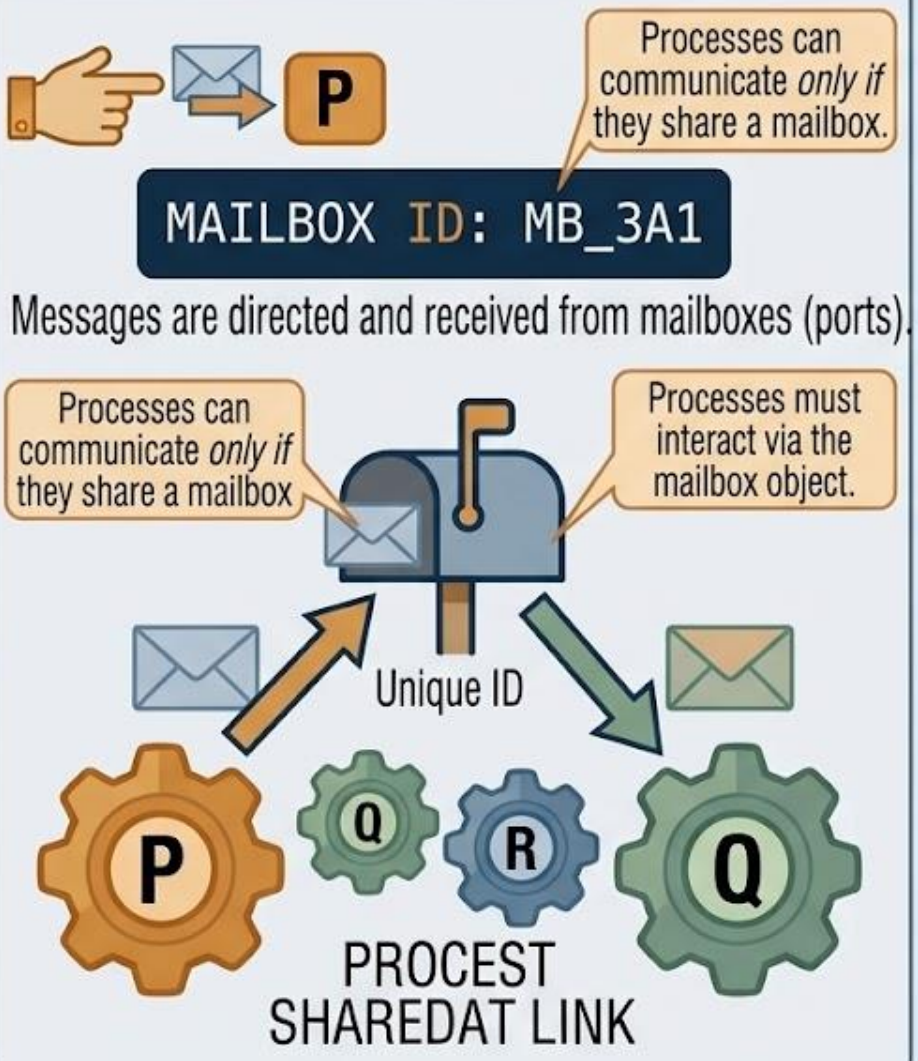


4. The link may be unidirectional, but is usually bi-directional.

Typically, it allows full-duplex message exchange.

IPC: INDIRECT COMMUNICATION (Mailboxes & Ports)

THE CONCEPT: SHARED MAILBOXES



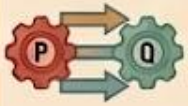
PROPERTIES OF THE COMMUNICATION LINK



1. Link established only if processes share a common mailbox.
Creation of a mailbox allows links to exist between all shared processes.



2. A link may be associated with many processes.
Many processes can send to or receive from the *same* mailbox.



3. Each pair of processes may share several communication links.
A pair (e.g., P and Q) can use different mailboxes simultaneously for different communication flows.



4. Link may be unidirectional or bi-directional. The capability is determined by the specific mailbox configuration.

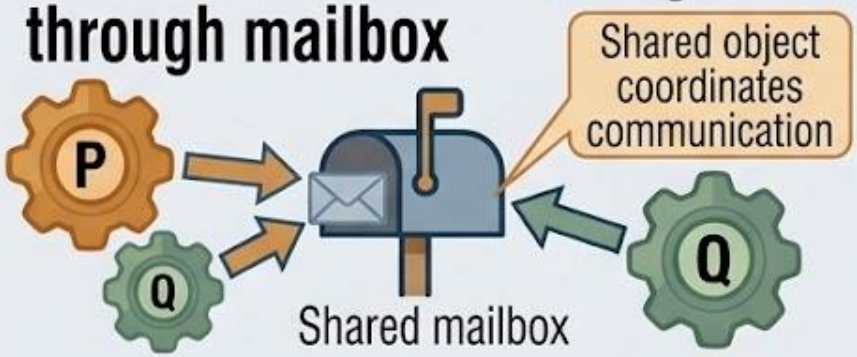
IPC: INDIRECT COMMUNICATION CONTINUED (Operations & Primitives)

MAILBOX OPERATIONS

- **Create a new mailbox (port)**



- **Send and receive messages through mailbox**



- **Delete a mailbox**

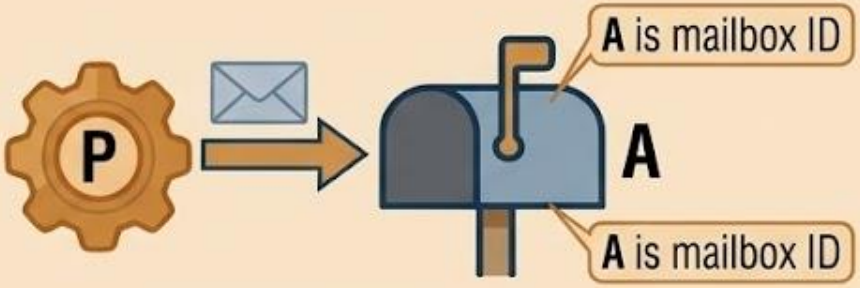


These system operations manage the communication infrastructure.

COMMUNICATION PRIMITIVES

`send(A, message)`

send a message to mailbox **A**



`receive(A, message)`

receive a message from mailbox **A**

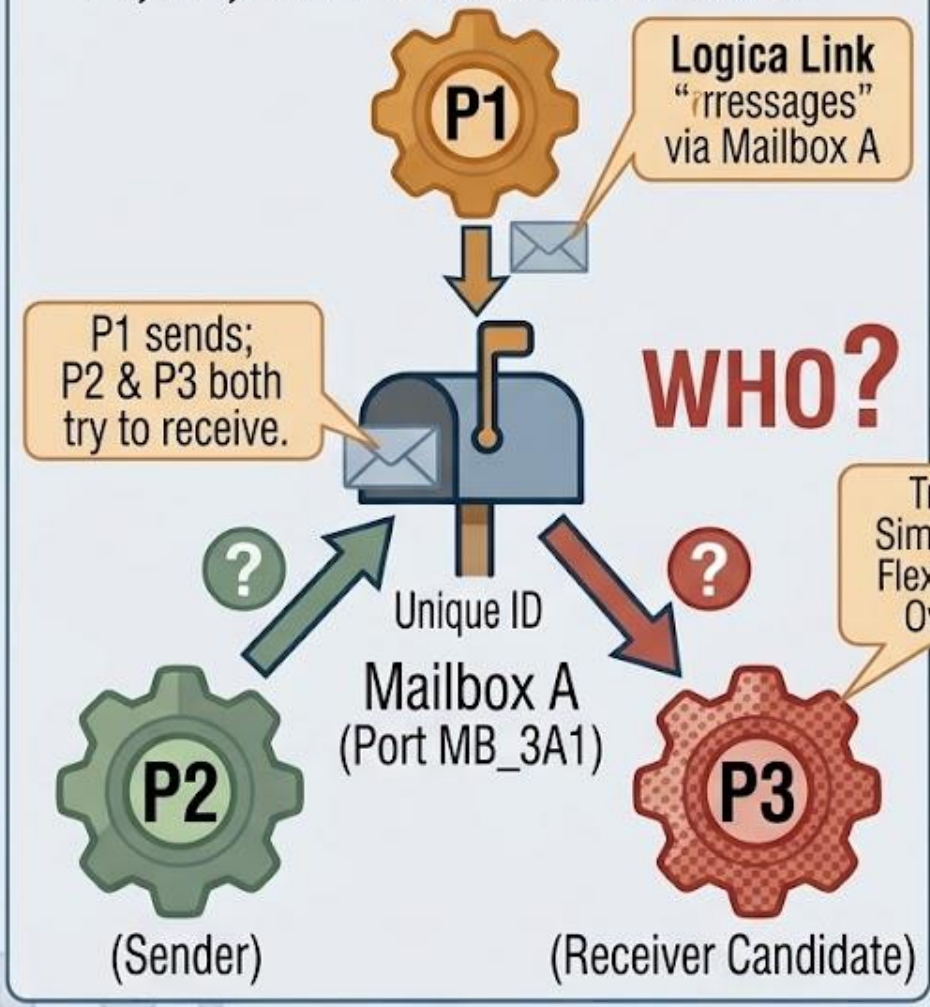


Primitives use mailbox IDs for addressing.

IPC: INDIRECT COMMUNICATION (Cont.) - Mailbox Sharing Scenario

THE PROBLEM: Who Gets the Message?

- **P1, P2, and P3 share mailbox A.**



SOLUTIONS & TRADE-OFFS

1. Allow link to be associated with *at most* two processes.

Less flexible sharing



Creation of a mailbox limits shared processes. Restricts sharing capability but simplifies routing.

2. Allow only one process at a time to execute a receive operation.



Many processes can send/receive from *same* mailbox, but must coordinate. Requires system synchronization (monitors/ semaphores) to select unique receiver.

3. Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.



A pair (e.g., P and Q) can use different mailboxes simultaneously. System handles the logic, but adds notification overhead.

WHERE EXACTLY IS IT STORED? (Message Storage in Indirect IPC)

KERNEL SPACE (RAM)

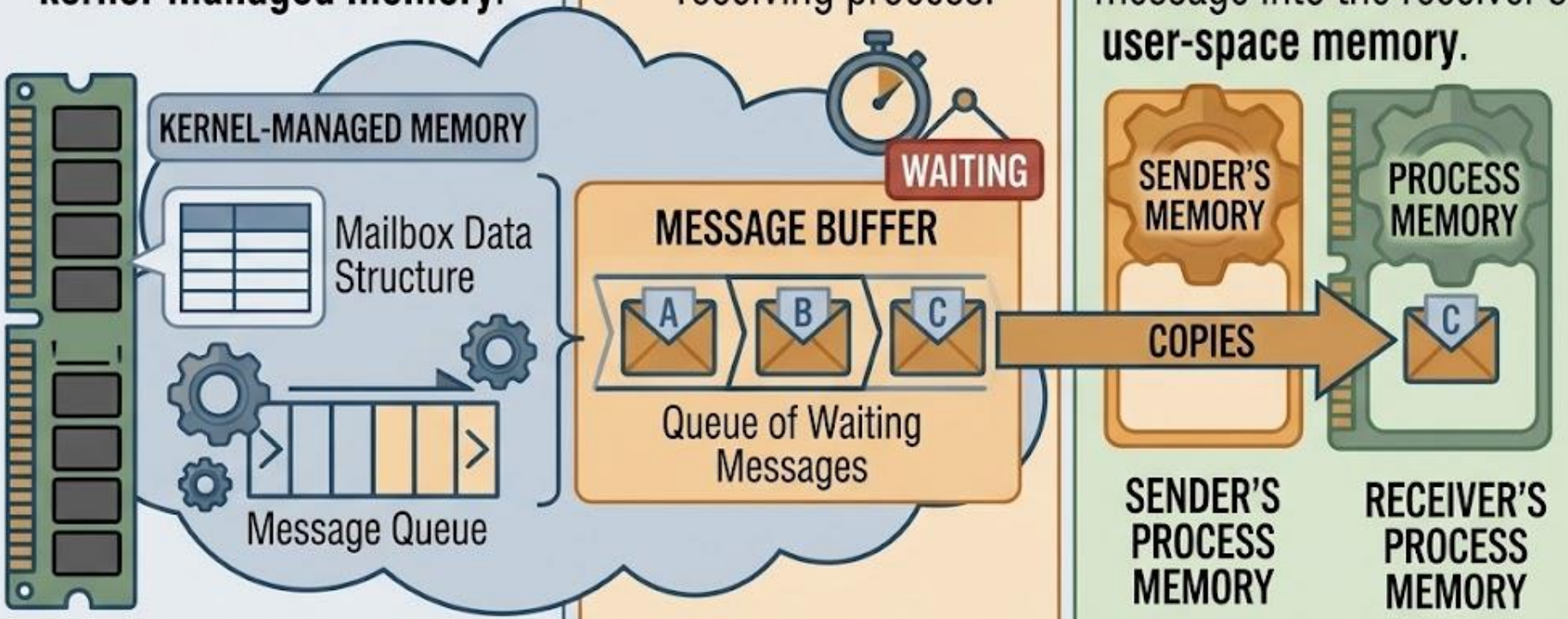
The operating system maintains the **mailbox data structure** and message queue in **kernel-managed memory**.

MESSAGE BUFFER

The messages may be stored in kernel memory while **waiting** for the receiving process.

PROCESS MEMORY

When a process receives a message, the operating system typically copies the message into the receiver's **user-space memory**.



Tradeoff: Overhead of system call and copy operation vs. process address space isolation

2. Key Differeces KEY DIFFERENCES: IPC MODELS

Feature	Mailbox (Message Passing)	Shared Buffer (Shared Memory)
Communication model	Message passing P → Q	Shared memory P & Q
Data access	Through send/receive operations	Direct memory access Direct pointer read/ write to memory
Typical storage	kernel Kernel-managed queue	Shared region in RAM
Synchronization	Often coordinated by IPC mechanism; blocking may be provided OS coordination	Usually explicitly coordinated by processes Explicit code synchronizing by processes
Performance	May involve system calls and data copying copy Slow system call	Often faster for large data transfers
Main concern	Queue capacity and message handling	Race conditions and data consistency

SYNCHRONIZATION: SEND() & RECEIVE() PRIMITIVES

SYNCHRONIZATION OVERVIEW



Communication takes place through **send()** and **receive()** calls.

There are different design options for implementing each primitive.

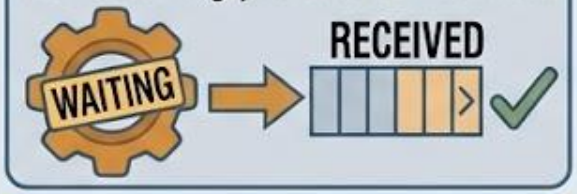
Message passing can be blocking (synchronous) or nonblocking (asynchronous).

DESIGN CHOICES ⇨ IMPLEMENTATION METHODS

SEND() BEHAVIORS (SENDER SIDE)

BLOCKING SEND (Synchronous)

The sending process is blocked until the message is received by the receiving process or mailbox.



NONBLOCKING SEND (Asynchronous)

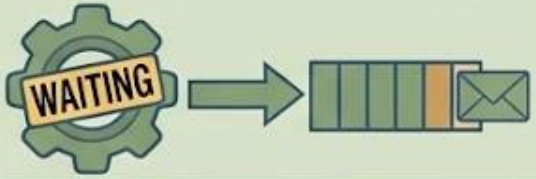
The sending process sends the message and resumes operation.



RECEIVE() BEHAVIORS (RECEIVER SIDE)

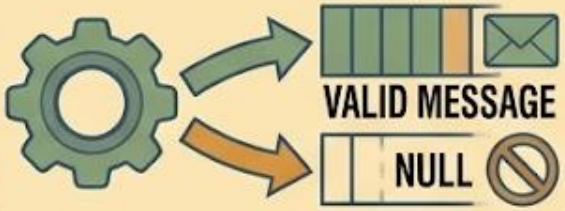
BLOCKING RECEIVE (Synchronous)

The receiver blocks until a message is available.



NONBLOCKING RECEIVE (Asynchronous)

The receiver retrieves either a **valid** message or a null.





Synchronization

Message passing may be either blocking or non-blocking

- **Blocking** is considered **synchronous**
  - **Blocking send** -- the sender is blocked until the message is received
  - **Blocking receive** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking send** -- the sender sends the message and continue
  - **Non-blocking receive** -- the receiver receives:
    - A valid message, or
    - Null message
- Different combinations possible
  - If both send and receive are blocking, we have a **rendezvous**





Producer-Consumer: Message Passing

- Producer

```
message next_produced;
while (true) {
  /* produce an item in next_produced */

  send(next_produced);
}
```

- Consumer

```
message next_consumed;
while (true) {
   receive(next_consumed)

  /* consume the item in next_consumed */
}
```

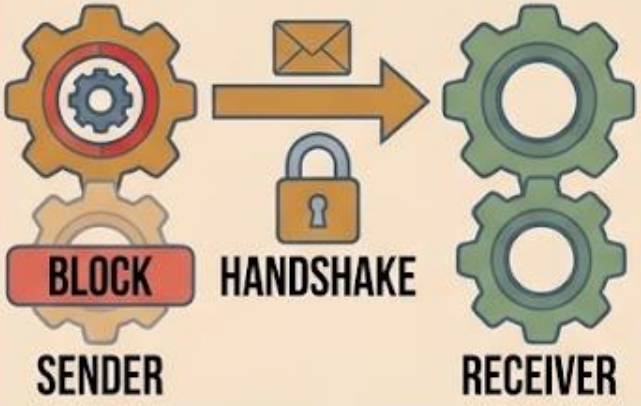


SYNCHRONIZATION: BUFFERING OPTIONS FOR MESSAGE QUEUES

0 ZERO CAPACITY (No Buffering)

The queue has a maximum length of zero; the link cannot have waiting messages.

The sender ***must block*** until until the recipient receives the message.



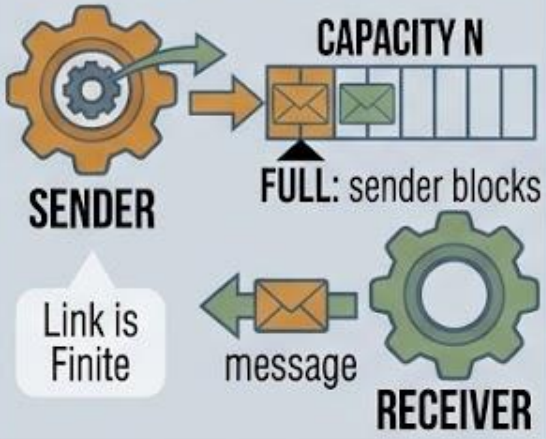
BOUNDED CAPACITY (Finite Buffering)



The queue has a finite length 'n'; at most 'n' messages can reside in it.

If not full, message is placed in queue, and sender continues.

Capacity is finite; if full, sender ***must block***.

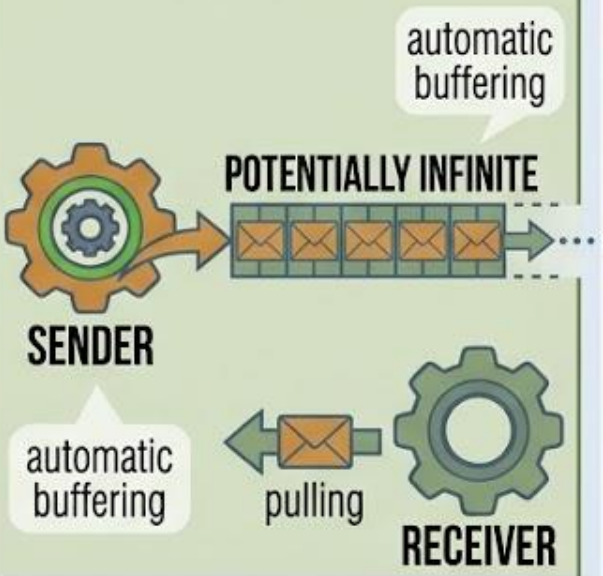


UNBOUNDED CAPACITY (Automatic Buffering)



The queue's length is potentially infinite; any number of messages can wait.

The sender ***never blocks***.



Summary: Performance Tradeoffs and Design Choices ⇨ Zero buffering (synchronization handshake) | Finite buffering (capacity limit) | Infinite buffering (never blocks).

IPC EXAMPLE: POSIX SHARED MEMORY (SYSTEM CALLS & FLOW)

STEP 1: INITIAL CREATION & SETUP (Producer)

Initial Setup

shm_open() - Create or open shm object.

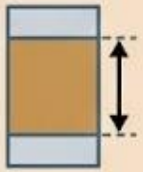
```
shm_fd = shm_open(name,
    O_CREAT | O_RDWR, 0666);
```

Create memory segment, name it, define permissions.

ftruncate() - Define size.

```
ftruncate(shm_fd, 4096);
```

Set size of the shared memory object object (e.g., 4096 bytes).



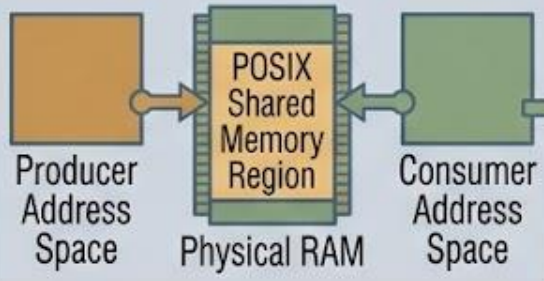
STEP 2: MEMORY MAPPING (Shared Memory)

Memory Mapping (Shared Region)

mmap() - Map object to memory.

```
void *ptr = mmap(0, 4096,
  PROT_READ | PROT_WRITE,
  MAP_SHARED, shm_fd, 0);
```

Memory-map a pointer (**ptr**) to the shared memory object, enabling direct access.



STEP 3: READING & WRITING (Data Transfer)

Reading/Writing

PRODUCER

Write data

```
sprintf(ptr, "%s", message);
```

CONSUMER

shm_open() - Open shm object.

mmap() - Map object to memory.

Read data

```
printf("%s", (char *)ptr);
```

Performance Tradeoffs ⇨ Fast direct memory access (no data copies, speed) vs. Complex explicit coordination (race conditions, synchronization).



IPC POSIX Producer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* strings written to shared memory */
const char *message_0 = "Hello";
const char *message_1 = "World!";

/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

    /* create the shared memory object */
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

    /* configure the size of the shared memory object */
    ftruncate(shm_fd, SIZE);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    /* write to the shared memory object */
    sprintf(ptr,"%s",message_0);
    ptr += strlen(message_0);
    sprintf(ptr,"%s",message_1);
    ptr += strlen(message_1);

    return 0;
}
```





IPC POSIX Consumer

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    /* read from the shared memory object */
    printf("%s",(char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

    return 0;
}
```



IPC EXAMPLE: MACH communication (system calls & ports)

KEY CHARACTERISTICS

Message Based
Communication is entirely message-based.

System Calls
Even system calls are implemented as messages.

Tasks & Ports
Each task gets two initial ports: Kernel & Notify.

MACH_MSG()

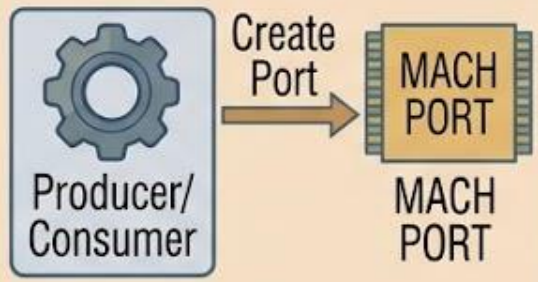
mach_msg() System Call

The single function used for sending and receiving messages.

```
mach_msg(msg_header, option,
                send_size, ...);
```

PORT ALLOCATION
mach_port_allocate()

Create a port




```
mach_port_allocate(task_port,
   MACH_PORT_RIGHT_RECEIVE,
   &port_name);
```

Primary function to create a new communication port.

FLEXIBLE SEND/RECEIVE OPTIONS (MAILBOX FULL)

Wait Options
(mailbox full)

****WAIT INDEFINITELY**

Block and wait until space is free.

****WAIT N MILLISECONDS**

Wait up to 'n' ms, then timeout.

****RETURN IMMEDIATELY****

Do not wait; return status indicating full.

****TEMPORARILY CACHE A MESSAGE****

Keep message in a special, limited cache.

Performance Tradeoffs ⇨ High flexibility for sender/receiver coordination (never/wait) vs. Complex implementation of system services (overhead).



Mach Messages

```
#include<mach/mach.h>

struct message {
        mach_msg_header_t header;
        int data;
};

mach port t client;
mach port t server;
```





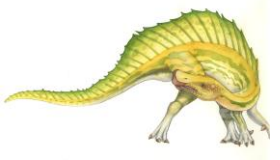
Mach Message Passing - Client

```
/* Client Code */

struct message message;

// construct the header
message.header.msgh_size = sizeof(message);
message.header.msgh_remote_port = server;
message.header.msgh_local_port = client;

// send the message
mach_msg(&message.header, // message header
   MACH_SEND_MSG, // sending a message
   sizeof(message), // size of message sent
   0, // maximum size of received message - unnecessary
   MACH_PORT_NULL, // name of receive port - unnecessary
   MACH_MSG_TIMEOUT_NONE, // no time outs
   MACH_PORT_NULL // no notify port
);
```





Mach Message Passing - Server

```
        /* Server Code */

struct message message;

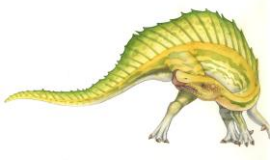
// receive the message
mach_msg(&message.header, // message header
   MACH_RCV_MSG, // sending a message
   0, // size of message sent
   sizeof(message), // maximum size of received message
   server, // name of receive port
   MACH_MSG_TIMEOUT_NONE, // no time outs
   MACH_PORT_NULL // no notify port
);
```





Examples of IPC Systems – Windows

- Message-passing centric via **advanced local procedure call (LPC)** facility
  - Only works between processes on the same system
  - Uses ports (like mailboxes) to establish and maintain communication channels
  - Communication works as follows:
    - The client opens a handle to the subsystem's **connection port** object.
    - The client sends a connection request.
    - The server creates two private **communication ports** and returns the handle to one of them to the client.
    - The client and server use the corresponding port handle to send messages or callbacks and to listen for replies.



IPC EXAMPLE: WINDOWS (ADVANCED LOCAL PROCEDURE CALL - ALPC)

ALPC Facility

Advanced Local Procedure Call (ALPC)
Message-passing centric via advanced local- call (LPC) facility.

Only works on same
Works only between processes on the same system.

Uses Ports (Mailboxes)
Uses ports to establish and maintain communication channels.

ALPC COMMUNICATION FLOW

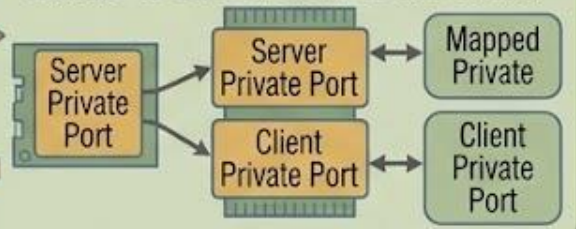
1. CLIENT SETUP
Client process

2. SERVER RESPONSE
Server process

3. MESSAGE EXCHANGE

1. The client opens a handle to the subsystem's connection port object.
2. The client sends a connection request.

3. The server creates two private communication ports and returns the handle to one of them to the client.



Uses Ports (Railboxes)
Mapped Private Memory Region

4. The client and server use the corresponding port handle to send messages or callbacks and to listen for replies.

Producer Client → **Consumer** Subsystem

```
sprintf(ptr, "%s", message);
```

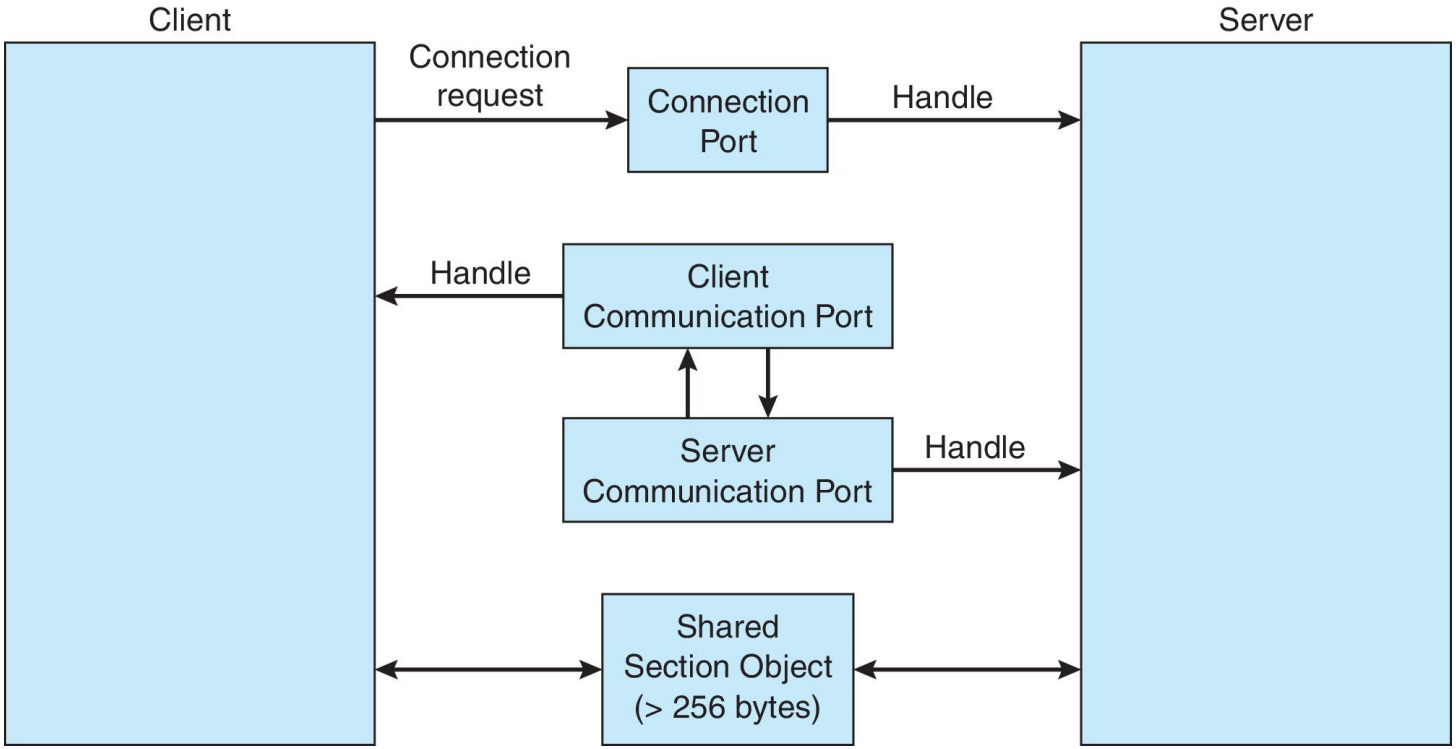
Fast direct message passing vs. kernel context switches and message queueing (synchronization overhead).

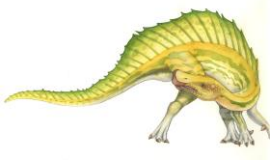
```
read_msg(private_port_handle, message);
```

Performance Tradeoffs ⇒ Efficient for frequent, small local communication (low overhead) vs. System service overhead (system calls, kernel switches).

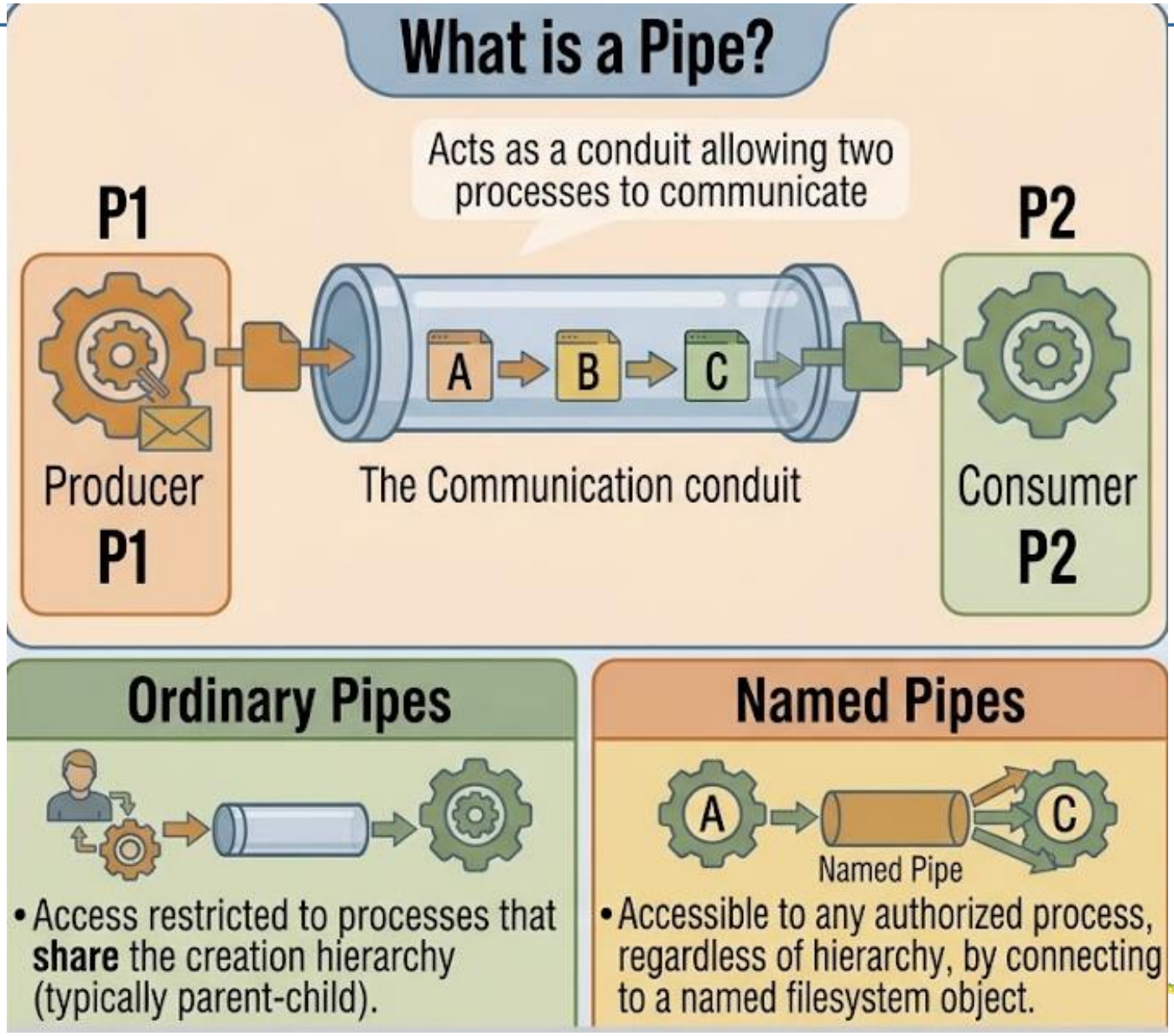


Local Procedure Calls in Windows





Pipes


What is a Pipe?
Acts as a conduit allowing two processes to communicate
P1
P2
A
B
C
Producer
P1
The Communication conduit
Consumer
P2
Ordinary Pipes
• Access restricted to processes that share the creation hierarchy (typically parent-child).
Named Pipes
A
C
Named Pipe
• Accessible to any authorized process, regardless of hierarchy, by connecting to a named filesystem object.

IPC CONCEPTS: PIPES (Design and Usage Issues)

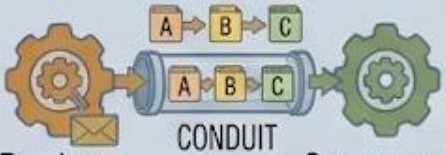
Acts as a conduit allowing two processes to communicate



Example: shm_open(), mmap() for direct access.

1. IS COMMUNICATION UNIDIRECTIONAL OR BIDIRECTIONAL?

UNIDIRECTIONAL (One Way)



Data flows only from P1→P2. →. Requires multiple pipes for two-way flow.

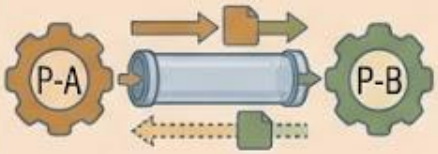
BIDIRECTIONAL (Two Way)



Data can flow both ways (A ↔ B) through a single or separate channels.

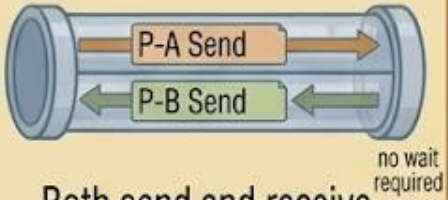
2. IN TWO-WAY COMMUNICATION, HALF OR FULL-DUPLEX?

HALF-DUPLEX



Can both send and receive, but not simultaneously. Must coordinate turns.

FULL-DUPLEX



Both send and receive simultaneously. Continuous two-way communication.

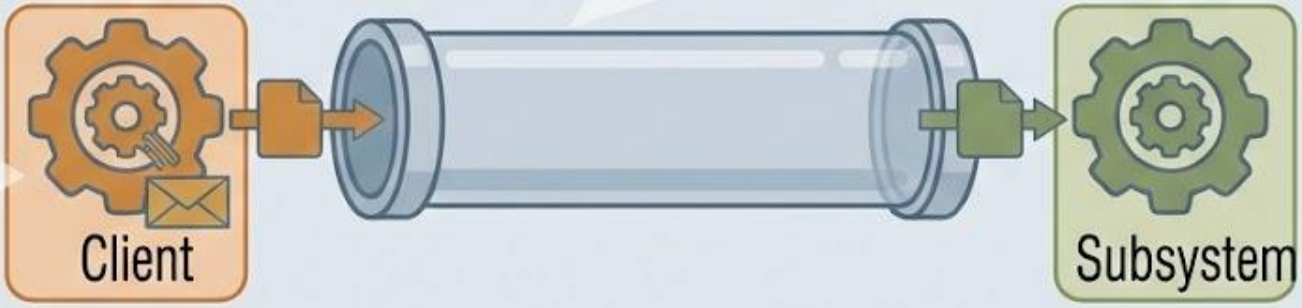
Summary of Key Design Choices and Tradeoffs ➡ Unidirectional vs. Bidirectional | Half-duplex vs. Full-duplex | Performance and complexity differences.

IPC CONCEPTS: PIPES (Design and Usage Issues)

Acts as a conduit allowing two processes to communicate

Producer

Consumer



Example: shm_open(), mmap() for direct access.

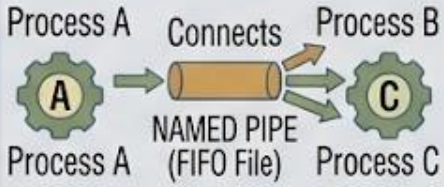
3. MUST THERE EXIST A RELATIONSHIP (E.G., PARENT-CHILD) BETWEEN THE COMMUNICATING PROCESSES?

RELATIONSHIP REQUIRED



Typically requires processes that share creation hierarchy (e.g., for Unnamed/Ordinary Pipes).

NO RELATIONSHIP REQUIRED



Any authorized process can connect by opening a named object (e.g., in a filesystem).

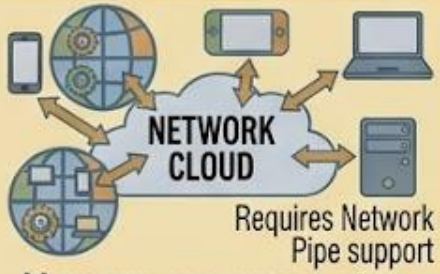
4. CAN THE PIPES BE USED OVER A NETWORK?

LOCAL SCOPE (Same System Only)



Communication limited to processes on the same system.

NETWORK SCOPE (Across Systems)



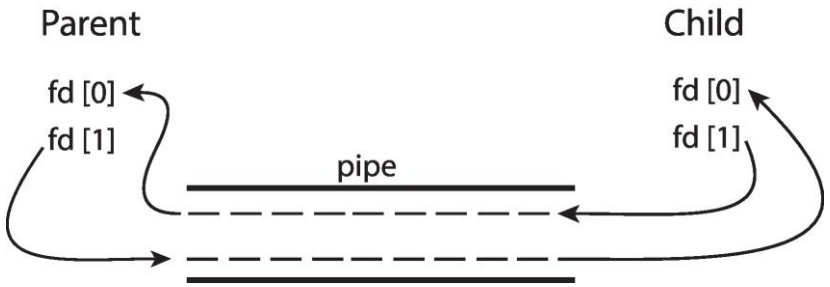
Messages can cross system boundaries to remote machines.

Summary of Key Design Choices and Tradeoffs ➡ Relationship vs. No Relationship (Anonymous vs. Named) | Local vs. Network Capability.

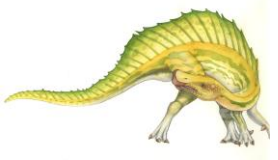


Ordinary Pipes

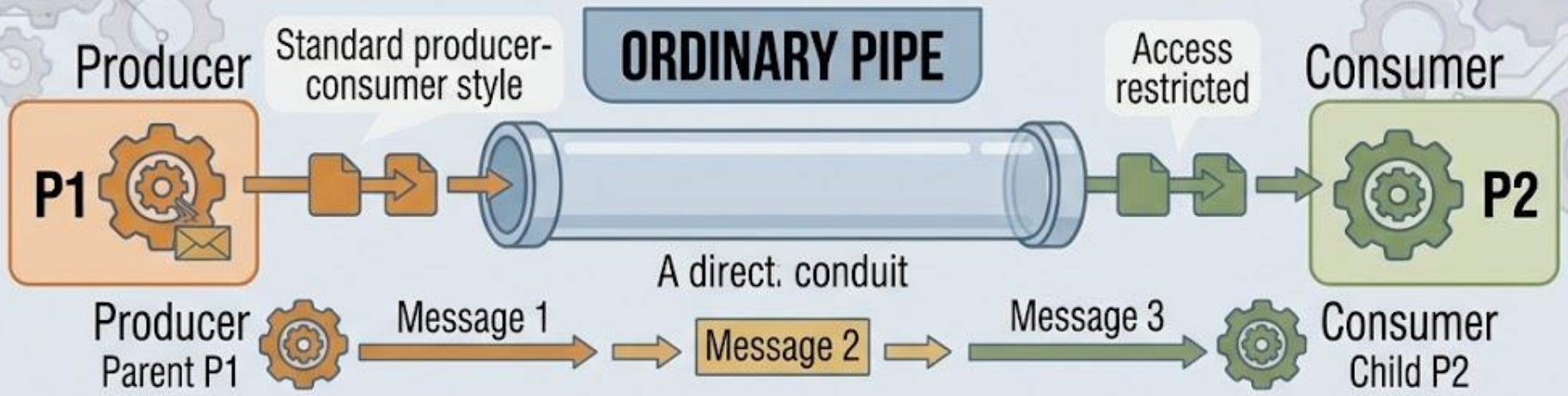
- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Ordinary pipes are therefore unidirectional
- Require parent-child relationship between communicating processes



- Windows calls these **anonymous pipes**



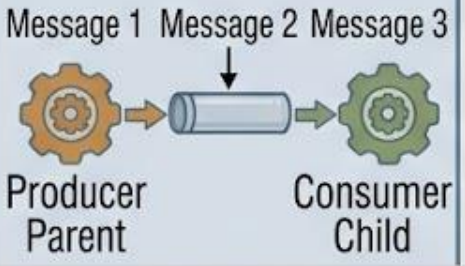
IPC CONCEPTS: ORDINARY PIPES (Parent-Child Communication)



**COMMUNICATION STYLE



Standard producer-consumer style communication.



**DATA ACCESS & FLOW

READ/WRITE

Producer writes to one end (write-end).

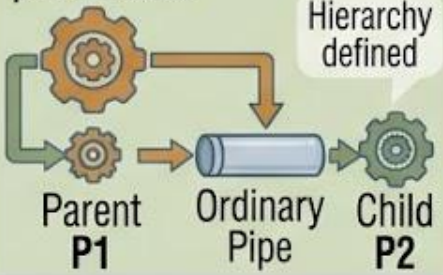
Consumer reads from the other end (read-end).

Ordinary pipes are therefore **UNIDIRECTIONAL** (one-way flow).



**PROCESS RELATIONSHIP

REQUIRE PARENT-CHILD RELATIONSHIP between communicating processes.



*LIMITATIONS



Typical limitations: Anonymous/unnamed, scope limited to process tree, temporary.



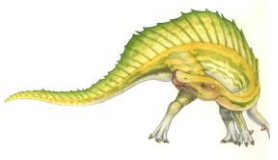
Anonymous: Cannot be accessed by unrelated processes.

Summary: Specific use cases for unidirectional, local, process-tree communication.

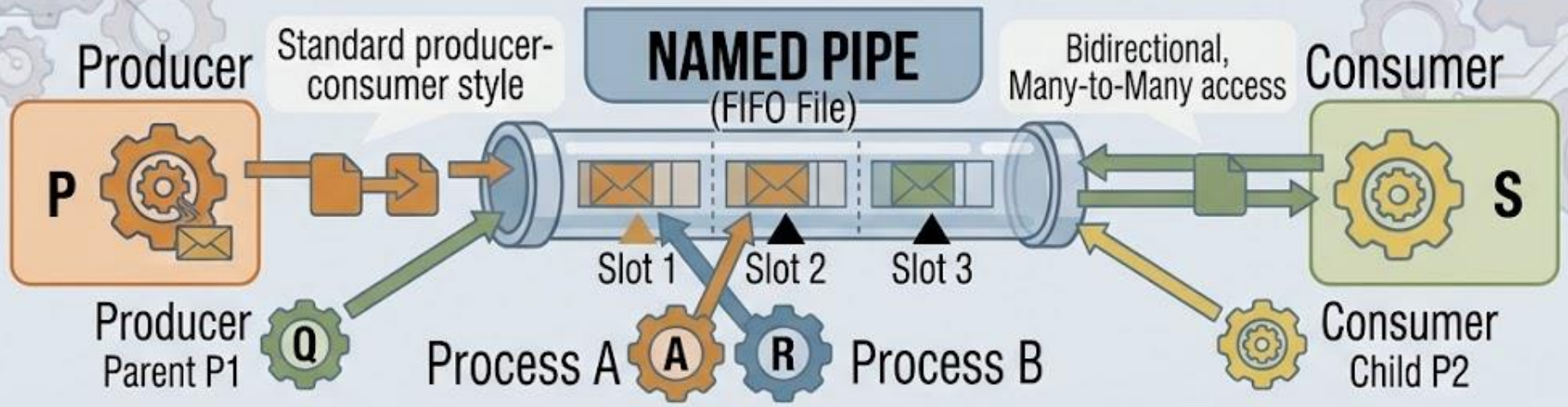


Named Pipes

- Named Pipes are more powerful than ordinary pipes
- Communication is bidirectional
- No parent-child relationship is necessary between the communicating processes
- Several processes can use the named pipe for communication
- Provided on both UNIX and Windows systems



IPC CONCEPTS: NAMED PIPES (FIFO)



**COMMUNICATION DIRECTION

Provided on both UNIX and Windows systems.



UNIX (FIFO files) & **WINDOWS** (\\.\pipe\)

**PROCESS RELATIONSHIP

NO PARENT-CHILD RELATIONSHIP is necessary between the communicating processes.

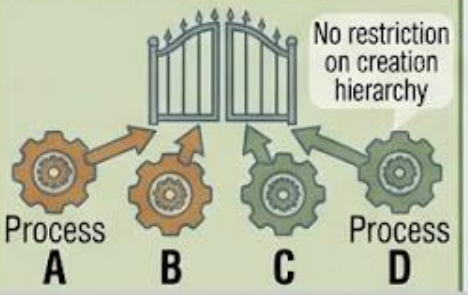
Process A

Any authorized process can connect by opening a named file object.

Process B

**PROCESS ACCESS

SEVERAL PROCESSES can use the named pipe for communication.



*COMPARISON vs Ordinary Pipe

Ordinary Pipe ( ): Unidirectional, Parent-Child only, local.

vs

vs **Named Pipe**: Bidirectional, any relationship, shared, network capable (some features).

Summary: Efficient for multi-process local/network communication, requires filesystem/named space.

COMPARISON OF INTER-PROCESS COMMUNICATION (IPC) MECHANISMS

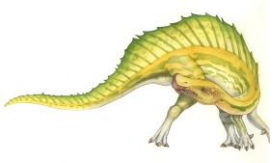
Feature	PIPE	MAILBOX / MESSAGE QUEUE	SHARED MEMORY	Context & Example Use Cases
Communication method	Data flows through a pipe	Processes send and receive messages	Processes directly access a common memory region	Context & Example Use case: thx/locatt
Data handling	Continuous byte stream	Separate messages	Shared data structures or buffers	Context & array memory rication diagrams
Memory management	OS Buffered pipe	OS OS typically manages the message queue	OS OS creates/maps the shared region; processes access it directly	
Communication style	Usually unidirectional	Message-based; may be one-way or two-way	Usually supports two-way access	
Process relationship	P C Anonymous pipes commonly connect related processes	Can connect related or unrelated processes	Can connect related or unrelated processes	
Synchronization	OS manages pipe access and blocking; application coordination may still be needed callout	OS manages message-queue operations	Processes generally need mutexes, semaphores, or other synchronization	Processes mutex/ured direct control
Performance	Moderate; data passes through pipe operations	Moderate; messages involve OS operations	Often fastest for large data transfers	Fastest, with large data packets
Example	P C Parent sends output to child	A M Process A sends a message to mailbox M	A B Two processes access the same shared array	Context data/ process region; with mopet memory array

Operating Systems: Inter-Process Communication (IPC) Concepts (Comparison Guide)



Communications in Client-Server Systems

- Sockets
- Remote Procedure Calls

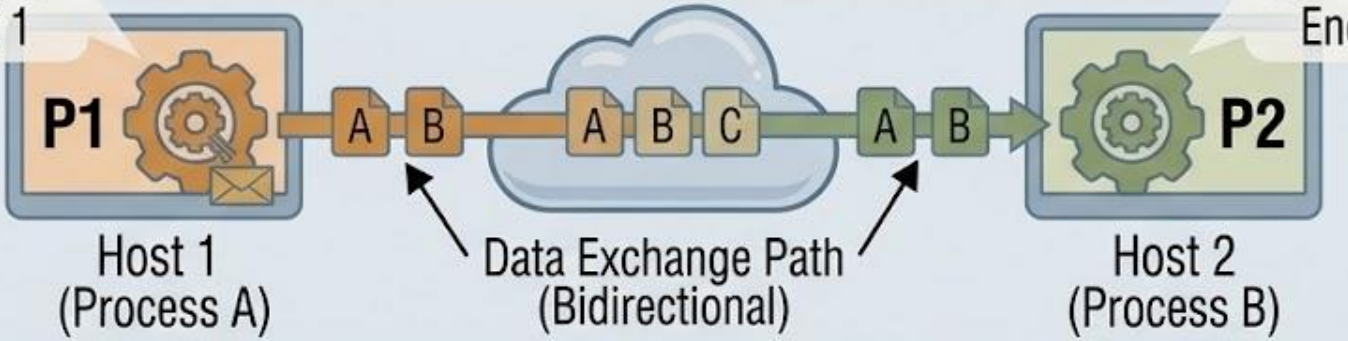


IPC CONCEPTS: THE ROLE OF A SOCKET

A Socket: The Communication Endpoint

SOCKET: Communication Endpoint 1

SOCKET: Communication Endpoint 2



**SOCKET DEFINITION & ADDRESSING

IP ADDR + : PORT NUM

A **socket** in IPC is a communication endpoint that allows two processes tos to exchange data.

Host: 192.168.1.100 Port: 8080

SOCKET: 192.168.1.100:8080

**DATA HANDLING & SYNCHRONIZATION

Provided with reliable and connection-oriented protocol features.

Synchronization and blocking managed by the system.

Send_data(socket, data) → Queue → Receive_data(socket, data)

Reliable Delivery

Implicit Sync

*COMPARISON vs Other IPC

Ordinary Pipe (): Unidirectional, Parent-Child only, local.

vs

Named Pipe: Bidirectional, any relationship, shared, network capable

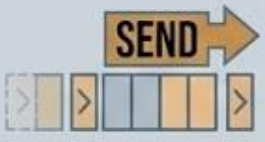
Shared Memory (): Fast, concurrent access, synchronization required.

IPC EXAMPLE: THE ROLE OF A SOCKET IN A WEBSITE VISIT

*1. BROWSER TO SERVER REQUEST



1. Your browser creates or **socket** to communicate with the web server.

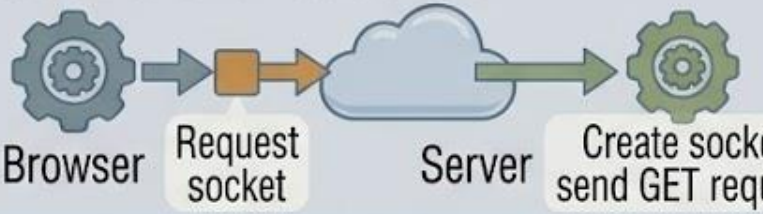


2. The socket sends a **request** over the network.



*1. BROWSER TROWSE FLOW

1. Your **browser** creates or uses a **socket** to netinort the web server.



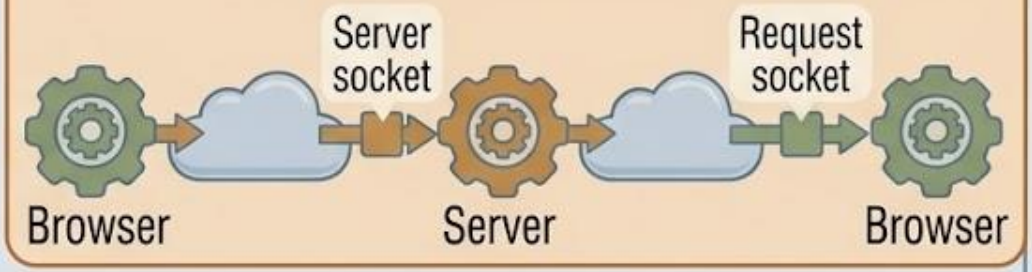
*2. SERVER RESPONSE & DATA FLOW



3. The server receives the request through its socket.



4. The **server** sends the webpage data **back** through the communication connection.



*3. LOCAL IPC ALTERNATIVE



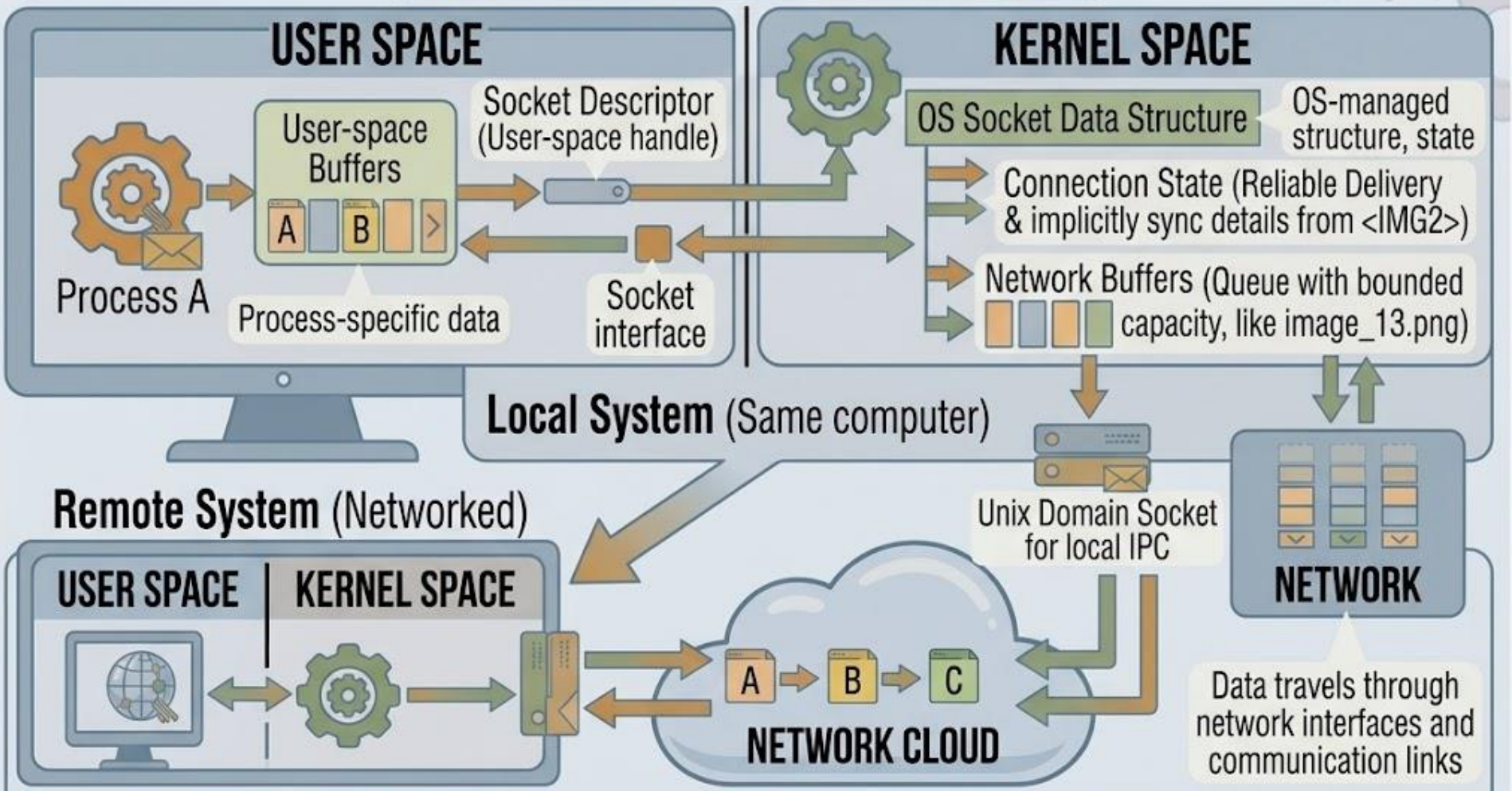
Sockets can also communicate between **processes** on the same computer, for example through Unix domain sockets.



Summary: Efficient connection-oriented communication, adaptable to local or networked scope.

IPC CONCEPTS: WHERE DOES A SOCKET RESIDE IN MEMORY?

A Socket: The Logical Endpoint



Summary: Application handles (user space) mapped to OS structures and buffers (kernel space), spanning to physical network interfaces.

IPC MECHANISM COMPARISON: SOCKET vs. PIPE vs. MAILBOX

	SOCKET (Endpoint)	PIPE (Conduit)	MAILBOX (Message Queue)
Communication	Same or different computers	Usually same computer	Usually same computer in OS IPC
Data model	Stream or datagrams Bounded capacity datagram	Byte stream	Messages Message Buffers Object
Typical use	Client-server applications	Related processes exchanging data	Processes exchanging queued messages
Example	Browser communicating with web server Browser Server	Parent sending data to child Parent Child	Process sending a message to a service Process A mailbox object M

In one line: A socket is a communication endpoint that enables processes to exchange data locally or across a network.

IPC MECHANISM REVIEW: THREE TYPES OF SOCKETS

**1. CONNECTION-ORIENTED (TCP)

Provided with reliable and connection-oriented protocol features.

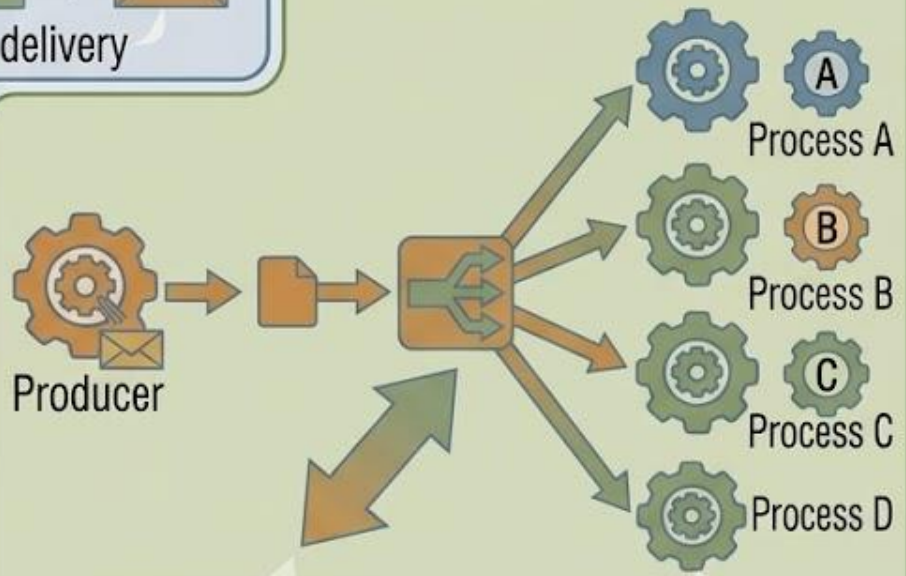
Legacied with rolls:

Establish Connection → Handshake → Bounded capacity datagram

A reliable delivery

3. MULTICASTSOCKET CLASS

A special type: data can be sent to multiple recipients.

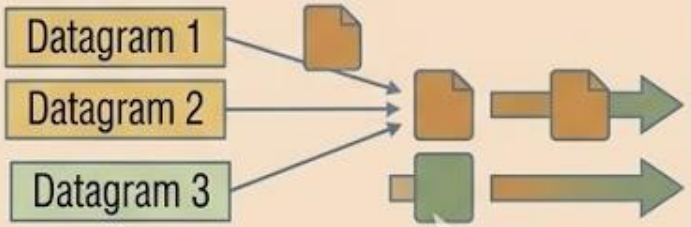


Multi-destination delivery: data to many recipients

IMAGE 3's Ordinary Pipe: No restriction on creation hierarchy for recipients

2. CONNECTIONLESS (UDP)

Basic, simpler datagram (UDP) feature.



Basic, simpliet datagram delivery

- one-way capability individual datagram message packets

Faster direct message passing, potentially unreliable delivery

Summary: Specific features for local/network communication needs → Zero buffering (synchronization handshake) | Finite buffering (capacity limit) | Infinite buffering (never blocks).



Sockets in Java

- Three types of sockets
  - **Connection-oriented** (**TCP**)
  - **Connectionless** (**UDP**)
  - **`MulticastSocket`** class– data can be sent to multiple recipients
- Consider this “Date” server in Java:

```
import java.net.*;
import java.io.*;

public class DateServer
{
   public static void main(String[] args) {
      try {
         ServerSocket sock = new ServerSocket(6013);

         /* now listen for connections */
         while (true) {
            Socket client = sock.accept();

            PrintWriter pout = new
             PrintWriter(client.getOutputStream(), true);

            /* write the Date to the socket */
            pout.println(new java.util.Date().toString());

            /* close the socket and resume */
            /* listening for connections */
            client.close();
         }
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```





Sockets in Java

The equivalent Date client

```
import java.net.*;
import java.io.*;

public class DateClient
{
   public static void main(String[] args) {
      try {
         /* make connection to server socket */
         Socket sock = new Socket("127.0.0.1",6013);

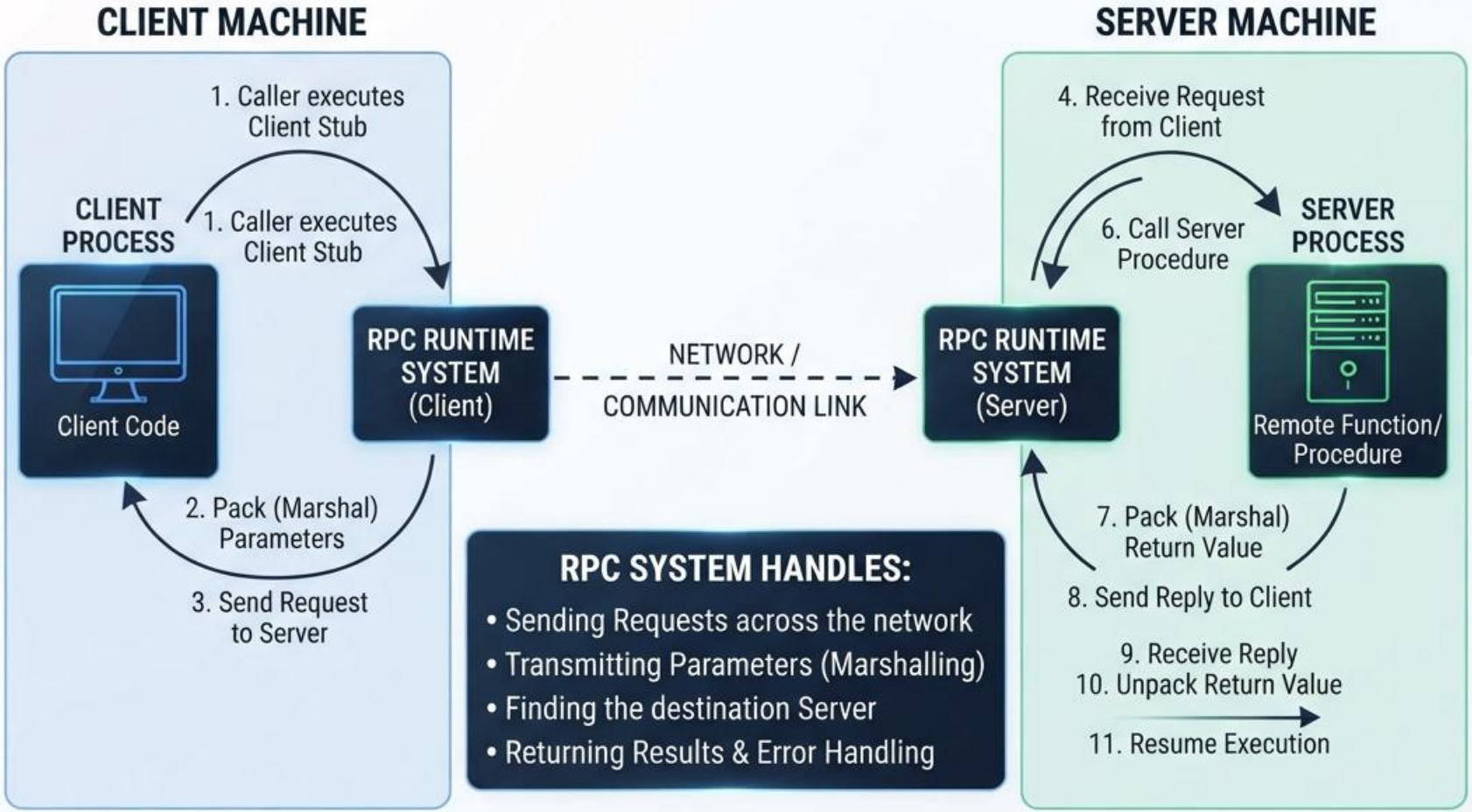
         InputStream in = sock.getInputStream();
         BufferedReader bin = new
            BufferedReader(new InputStreamReader(in));

         /* read the date from the socket */
         String line;
         while ( (line = bin.readLine()) != null)
            System.out.println(line);

         /* close the socket connection*/
         sock.close();
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```

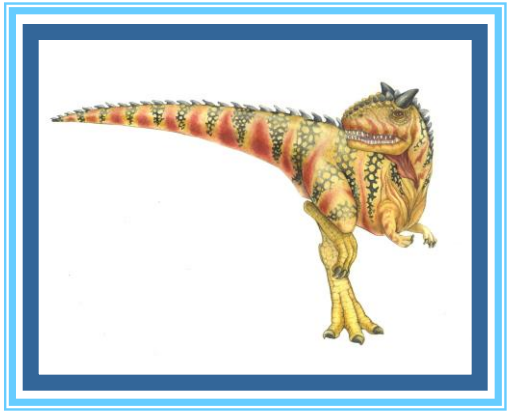


REMOTE PROCEDURE CALL (RPC) IN IPC



INTER-PROCESS COMMUNICATION MECHANISM

End of Chapter 3



Silberschatz, Galvin and Gagne ©2018